

Universidade do Minho
Escola de Engenharia

Comunicações por Computador

2026/2027

Trabalho Prático 1

Elementos do Grupo

Beatriz Sousa Martins - A109958

Joana Catarina Fernandes Rodrigues - A112407

Maria Ana Maciel de Aguiar de Antas Kopke - A110763

Margarida Cunha da Silva - A104357

Questões e Respostas

Parte I: Criação e caracterização da rede de testes

1.1 Criação de uma topologia de testes

a) Execute a simulação, e faça os testes necessários para poder responder a todas questões que se seguem

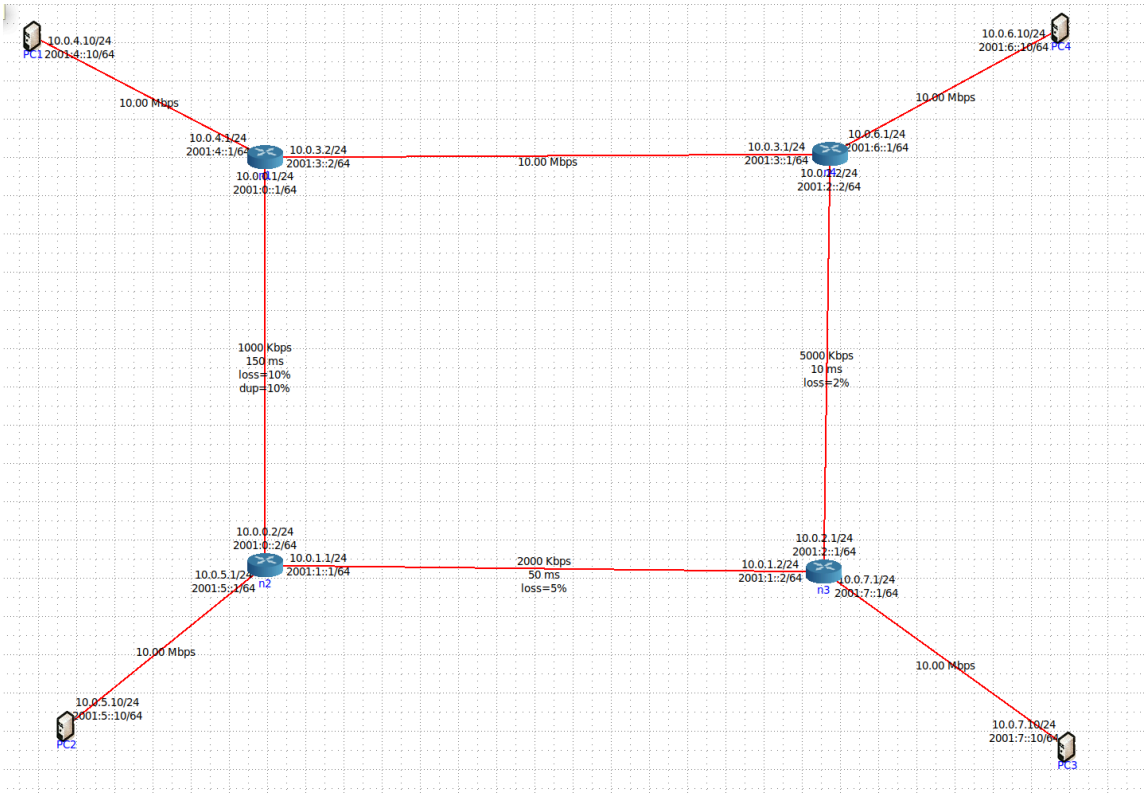


Figura 1 - Topologia

1.2 Testes elementares de conectividade

```
root@PC1:/tmp/pycore.46881/PC1.conf# traceroute -n 10.0.5.10
traceroute to 10.0.5.10 (10.0.5.10), 30 hops max, 60 byte packets
 1  10.0.4.1  0.220 ms  1.522 ms  1.487 ms
 2  10.0.0.2  302.185 ms  303.347 ms  304.062 ms
 3  10.0.5.10  305.047 ms  305.805 ms  306.366 ms
root@PC1:/tmp/pycore.46881/PC1.conf# traceroute -n 10.0.7.10
traceroute to 10.0.7.10 (10.0.7.10), 30 hops max, 60 byte packets
 1  10.0.4.1  0.242 ms  1.495 ms  1.464 ms
 2  10.0.0.2  302.010 ms *  303.637 ms
 3  10.0.1.2  405.473 ms  405.986 ms  407.613 ms
 4  10.0.7.10  408.400 ms *  409.921 ms
root@PC1:/tmp/pycore.46881/PC1.conf# traceroute -n 10.0.6.10
traceroute to 10.0.6.10 (10.0.6.10), 30 hops max, 60 byte packets
 1  10.0.4.1  0.245 ms  0.490 ms  0.457 ms
 2  10.0.3.1  0.776 ms  0.734 ms  0.914 ms
 3  10.0.6.10  1.361 ms  1.296 ms  1.618 ms
root@PC1:/tmp/pycore.46881/PC1.conf#
```

Figura 2 - Output do comando traceroute a partir do terminal do PC1

A análise da utilização do comando traceroute na figura acima demonstra que os pacotes percorrem por nós esperados entre os hosts, o que valida a configuração do encaminhamento.

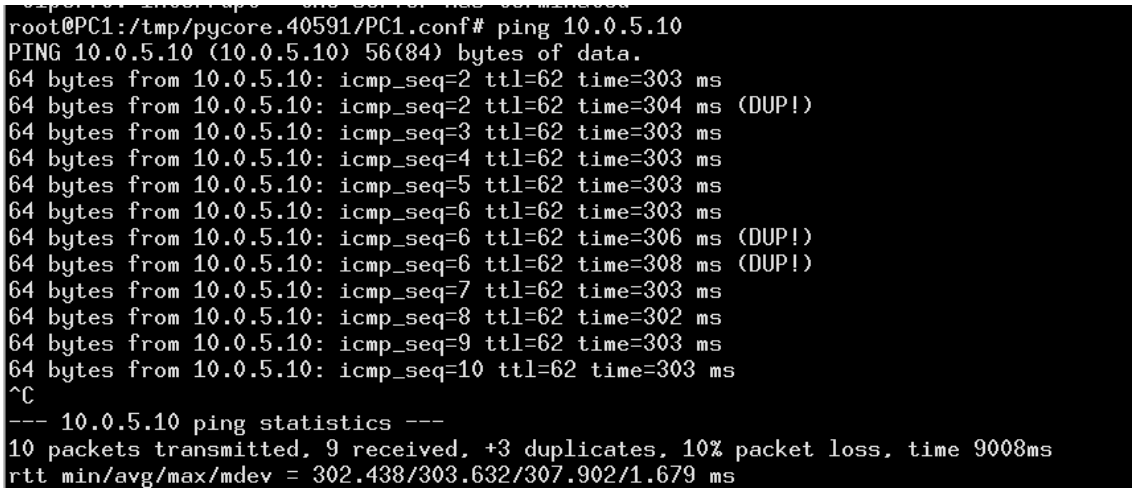


Figura 3 - Ping do PC1 para o PC2

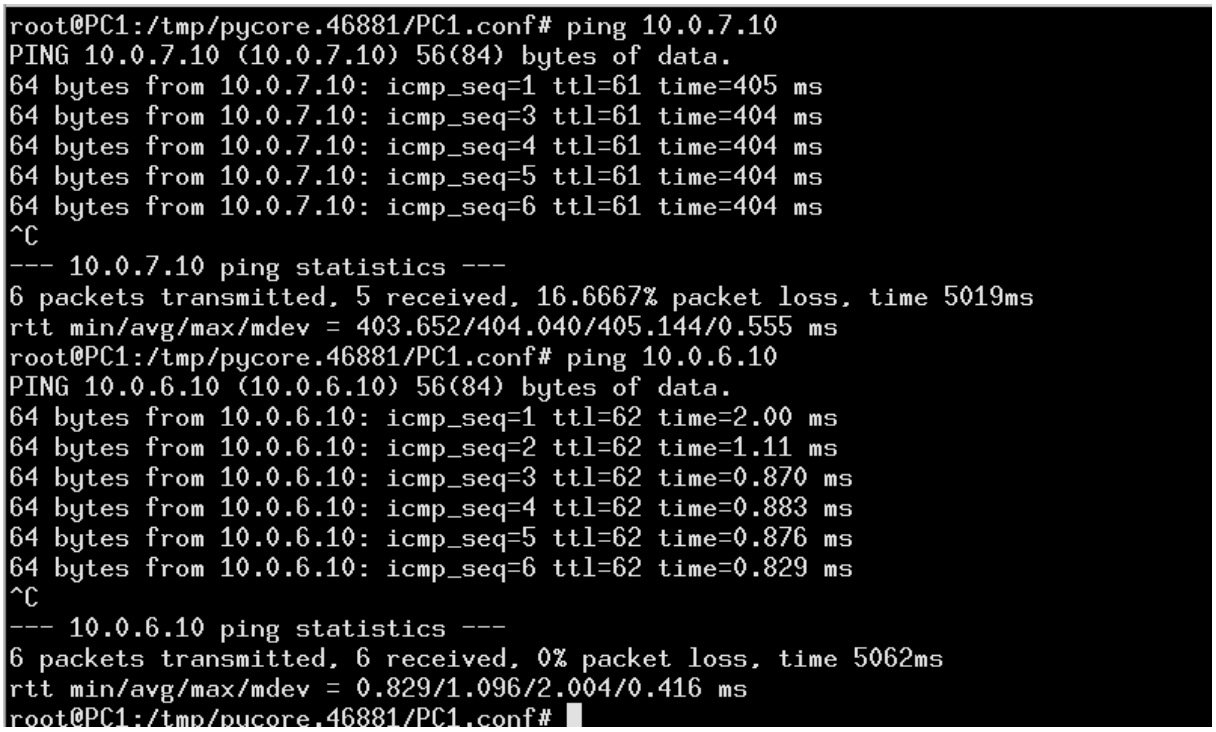


Figura 4 - Ping do PC1 para o PC3 3 do PC1 para o PC4, respetivamente

Com o comando ping (Figura 3 e 4) conseguimos medir os atrasos e verificar a conectividade entre dois hosts, além de indicar a perda de pacotes se houver falhas na comunicação, bem como indicar a duplicação de pacotes.

Os valores de perdas medidos não coincidem com os configurados por duas razões. A primeira é que a perda configurada se aplica a cada ligação e a cada sentido, pelo que a perda fim-a-fim se acumula ao longo da rota e é superior à de qualquer ligação isolada. No PC3, que atravessa as ligações n1–n2 (10%) e n2–n3 (5%), a perda esperada é por isso

bastante superior aos 5% da ligação n2–n3, e no PC2 é superior aos 10% da ligação n1–n2, porque tanto o pedido como a resposta podem perder-se.

A segunda razão é que a perda é gerada de forma aleatória, pelo que só converge para o valor esperado com um número elevado de pacotes. Com amostras pequenas, a percentagem varia muito entre testes, já que cada pacote perdido pesa vários pontos percentuais. Além disso, com RTT elevados (300 a 400 ms), o último pacote enviado pode ainda estar em trânsito quando o ping é interrompido, sendo contado como perdido sem ter sido. Por estes motivos, os valores obtidos devem ser interpretados como estimativas, e só se aproximam do esperado com testes mais longos.

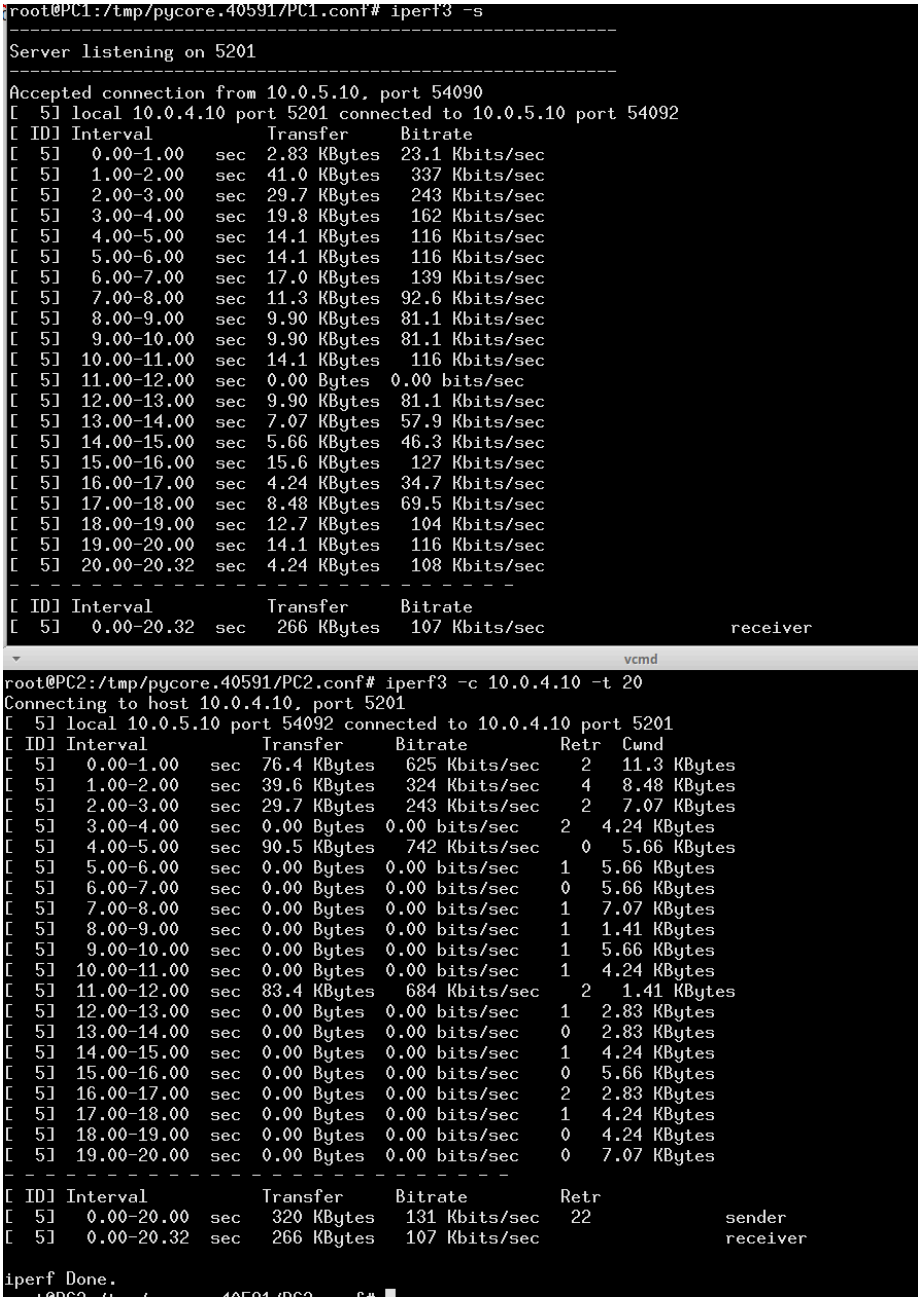


Figura 5 - Output do comando iperf mostrando o tráfego TCP entre os hosts PC1->PC2

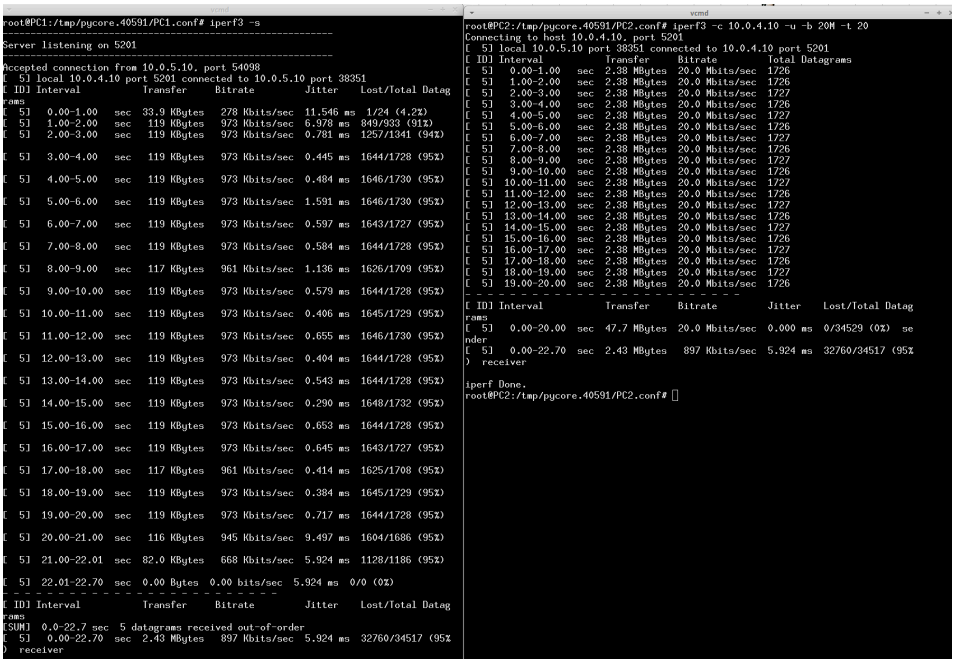
```
root@PC1:/tmp/pycore.46881/PC1.conf# iperf3 -c 10.0.7.10 -t 20
Connecting to host 10.0.7.10, port 5201
[  5] local 10.0.4.10 port 60898 connected to 10.0.7.10 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  76.4 KBytes   625 Kbits/sec    2   11.3 KBytes
[  5]   1.00-2.00   sec  53.7 KBytes   441 Kbits/sec    2   7.07 KBytes
[  5]   2.00-3.00   sec  0.00 Bytes  0.00 bits/sec    0   11.3 KBytes
[  5]   3.00-4.00   sec  0.00 Bytes  0.00 bits/sec    4   7.07 KBytes
[  5]   4.00-5.00   sec  0.00 Bytes  0.00 bits/sec    2   4.24 KBytes
[  5]   5.00-6.00   sec  0.00 Bytes  0.00 bits/sec    0   5.66 KBytes
[  5]   6.00-7.00   sec  0.00 Bytes  0.00 bits/sec    3   2.83 KBytes
[  5]   7.00-8.00   sec  84.8 KBytes   695 Kbits/sec    0   4.24 KBytes
[  5]   8.00-9.00   sec  0.00 Bytes  0.00 bits/sec    1   4.24 KBytes
[  5]   9.00-10.00  sec  0.00 Bytes  0.00 bits/sec    0   4.24 KBytes
[  5]  10.00-11.00  sec  0.00 Bytes  0.00 bits/sec    2   2.83 KBytes
[  5]  11.00-12.00  sec  0.00 Bytes  0.00 bits/sec    0   5.66 KBytes
[  5]  12.00-13.00  sec  0.00 Bytes  0.00 bits/sec    1   4.24 KBytes
[  5]  13.00-14.00  sec  0.00 Bytes  0.00 bits/sec    3   4.24 KBytes
[  5]  14.00-15.00  sec  0.00 Bytes  0.00 bits/sec    0   8.48 KBytes
[  5]  15.00-16.00  sec  0.00 Bytes  0.00 bits/sec    3   4.24 KBytes
[  5]  16.00-17.00  sec  0.00 Bytes  0.00 bits/sec    0   8.48 KBytes
[  5]  17.00-18.00  sec  0.00 Bytes  0.00 bits/sec    1   5.66 KBytes
[  5]  18.00-19.00  sec  84.8 KBytes   695 Kbits/sec    0   8.48 KBytes
[  5]  19.00-20.00  sec  0.00 Bytes  0.00 bits/sec    2   4.24 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-20.00  sec   300 KBytes   123 Kbits/sec   26             sender
[  5]   0.00-20.59  sec   182 KBytes  72.6 Kbits/sec                  receiver

iperf Done.
root@PC1:/tmp/pycore.46881/PC1.conf# iperf3 -c 10.0.6.10 -t 20
Connecting to host 10.0.6.10, port 5201
[  5] local 10.0.4.10 port 58522 connected to 10.0.6.10 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  1.47 MBytes  12.4 Mbits/sec    0   87.7 KBytes
[  5]   1.00-2.00   sec  1.55 MBytes  13.0 Mbits/sec    0    146 KBytes
[  5]   2.00-3.00   sec  1.49 MBytes  12.5 Mbits/sec    0    205 KBytes
[  5]   3.00-4.00   sec   954 KBytes  7.82 Mbits/sec    0    263 KBytes
[  5]   4.00-5.00   sec  1.74 MBytes  14.6 Mbits/sec    0    321 KBytes
[  5]   5.00-6.00   sec  1.43 MBytes  12.0 Mbits/sec    0    379 KBytes
[  5]   6.00-7.00   sec   827 KBytes  6.78 Mbits/sec    0    437 KBytes
[  5]   7.00-8.00   sec  1.80 MBytes  15.1 Mbits/sec    0    496 KBytes
[  5]   8.00-9.00   sec  1018 KBytes  8.34 Mbits/sec    0    554 KBytes
[  5]   9.00-10.00  sec  2.30 MBytes  19.3 Mbits/sec    0    612 KBytes
[  5]  10.00-11.00  sec  1.25 MBytes  10.5 Mbits/sec    0    699 KBytes
[  5]  11.00-12.00  sec  1.25 MBytes  10.5 Mbits/sec    0    872 KBytes
[  5]  12.00-13.00  sec  0.00 Bytes  0.00 bits/sec    0   1.06 MBytes
[  5]  13.00-14.00  sec  1.25 MBytes  10.5 Mbits/sec    0   1.29 MBytes
[  5]  14.00-15.00  sec  1.25 MBytes  10.5 Mbits/sec    0   1.56 MBytes
[  5]  15.00-16.00  sec  1.25 MBytes  10.5 Mbits/sec   66   1.56 MBytes
[  5]  16.00-17.00  sec  0.00 Bytes  0.00 bits/sec  189   1.23 MBytes
[  5]  17.00-18.00  sec  2.50 MBytes  21.0 Mbits/sec    0   1.23 MBytes
[  5]  18.00-19.00  sec  1.25 MBytes  10.5 Mbits/sec    0   1.29 MBytes
[  5]  19.00-20.00  sec  1.25 MBytes  10.5 Mbits/sec    0   1.39 MBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-20.00  sec  25.8 MBytes  10.8 Mbits/sec  255             sender
[  5]   0.00-21.23  sec  24.1 MBytes  9.52 Mbits/sec                  receiver

iperf Done.
```

Figura 6 - Output do comando iperf mostrando o tráfego TCP entre os hosts PC1->PC3 e PC1->PC4

Utilizando o comando iperf conseguimos medir a largura de banda e o desempenho de rede entre dois hosts.



```
root@PC1:/tmp/pycore.40591/PC1.conf# iperf3 -s
-----------------------------------------------------------
Server listening on 5201
-----------------------------------------------------------
Accepted connection from 10.0.5.10, port 54098
[  5] local 10.0.4.10 port 5201 connected to 10.0.5.10 port 38351
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  5]   0.00-1.00   sec  33.9 KBytes   278 Kbits/sec  11.546 ms  1/24 (4.2%)
[  5]   1.00-2.00   sec   119 KBytes   973 Kbits/sec  6.978 ms  849/933 (91%)
[  5]   2.00-3.00   sec   119 KBytes   973 Kbits/sec  0.701 ms  1257/1341 (94%)
[  5]   3.00-4.00   sec   119 KBytes   973 Kbits/sec  0.445 ms  1644/1728 (95%)
[  5]   4.00-5.00   sec   119 KBytes   973 Kbits/sec  0.484 ms  1646/1730 (95%)
[  5]   5.00-6.00   sec   119 KBytes   973 Kbits/sec  1.591 ms  1646/1730 (95%)
[  5]   6.00-7.00   sec   119 KBytes   973 Kbits/sec  0.597 ms  1643/1727 (95%)
[  5]   7.00-8.00   sec   119 KBytes   973 Kbits/sec  0.584 ms  1644/1728 (95%)
[  5]   8.00-9.00   sec   117 KBytes   961 Kbits/sec  1.136 ms  1626/1709 (95%)
[  5]   9.00-10.00  sec   119 KBytes   973 Kbits/sec  0.579 ms  1644/1728 (95%)
[  5]  10.00-11.00  sec   119 KBytes   973 Kbits/sec  0.406 ms  1645/1729 (95%)
[  5]  11.00-12.00  sec   119 KBytes   973 Kbits/sec  0.655 ms  1646/1730 (95%)
[  5]  12.00-13.00  sec   119 KBytes   973 Kbits/sec  0.404 ms  1644/1728 (95%)
[  5]  13.00-14.00  sec   119 KBytes   973 Kbits/sec  0.543 ms  1644/1728 (95%)
[  5]  14.00-15.00  sec   119 KBytes   973 Kbits/sec  0.290 ms  1648/1732 (95%)
[  5]  15.00-16.00  sec   119 KBytes   973 Kbits/sec  0.653 ms  1644/1728 (95%)
[  5]  16.00-17.00  sec   119 KBytes   973 Kbits/sec  0.645 ms  1643/1727 (95%)
[  5]  17.00-18.00  sec   117 KBytes   961 Kbits/sec  0.414 ms  1625/1708 (95%)
[  5]  18.00-19.00  sec   119 KBytes   973 Kbits/sec  0.384 ms  1645/1729 (95%)
[  5]  19.00-20.00  sec   119 KBytes   973 Kbits/sec  0.717 ms  1644/1728 (95%)
[  5]  20.00-21.00  sec   116 KBytes   945 Kbits/sec  9.497 ms  1604/1686 (95%)
[  5]  21.00-22.01  sec  82.0 KBytes   668 Kbits/sec  5.924 ms  1128/1186 (95%)
[  5]  22.01-22.70  sec  0.00 Bytes  0.00 bits/sec  5.924 ms  0/0 (0%)
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[SUM]  0.0-22.7 sec  5 datagrams received out-of-order
[  5]   0.00-22.70  sec  2.43 MBytes   897 Kbits/sec  5.924 ms  32760/34517 (95%
)  receiver
```

```
root@PC2:/tmp/pycore.40591/PC2.conf# iperf3 -c 10.0.4.10 -u -b 20M -t 20
Connecting to host 10.0.4.10, port 5201
[  5] local 10.0.5.10 port 38351 connected to 10.0.4.10 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   1.00-2.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   2.00-3.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   3.00-4.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   4.00-5.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   5.00-6.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   6.00-7.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   7.00-8.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   8.00-9.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   9.00-10.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  10.00-11.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  11.00-12.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  12.00-13.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  13.00-14.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  14.00-15.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  15.00-16.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  16.00-17.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  17.00-18.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  18.00-19.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  19.00-20.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  5]   0.00-20.00  sec  47.7 MBytes  20.0 Mbits/sec  0.000 ms  0/34529 (0%)  se
nder
[  5]   0.00-22.70  sec  2.43 MBytes   897 Kbits/sec  5.924 ms  32760/34517 (95%
)  receiver

iperf Done.
root@PC2:/tmp/pycore.40591/PC2.conf#
```

Figura 7 - Output do comando iperf mostrando o tráfego UDP entre os dois hosts PC1->PC2

```
root@PC1:/tmp/pycore.46881/PC1.conf# iperf3 -c 10.0.7.10 -u -b 20M -t 20
Connecting to host 10.0.7.10, port 5201
[  5] local 10.0.4.10 port 38927 connected to 10.0.7.10 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.04   sec  1.41 KBytes  11.2 Kbits/sec  1
[  5]   1.04-2.00   sec  4.77 MBytes  41.5 Mbits/sec  3451
[  5]   2.00-3.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   3.00-4.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   4.00-5.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   5.00-6.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   6.00-7.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   7.00-8.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   8.00-9.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   9.00-10.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  10.00-11.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  11.00-12.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  12.00-13.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  13.00-14.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  14.00-15.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  15.00-16.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  16.00-17.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  17.00-18.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  18.00-19.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  19.00-20.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-20.00  sec  47.7 MBytes  20.0 Mbits/sec  0.000 ms  0/34529 (0%)  sender
[  5]   0.00-25.72  sec  2.23 MBytes   729 Kbits/sec  9.703 ms  -1932750414/-1932750413 (0%)  receiver

iperf Done.
root@PC1:/tmp/pycore.46881/PC1.conf# iperf3 -c 10.0.6.10 -u -b 20M -t 20
Connecting to host 10.0.6.10, port 5201
[  5] local 10.0.4.10 port 39707 connected to 10.0.6.10 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  2.38 MBytes  20.0 Mbits/sec  1725
[  5]   1.00-2.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   2.00-3.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   3.00-4.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   4.00-5.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   5.00-6.00   sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]   6.00-7.00   sec  2.39 MBytes  20.0 Mbits/sec  1728
[  5]   7.00-8.00   sec  2.38 MBytes  20.0 Mbits/sec  1725
[  5]   8.00-9.00   sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]   9.00-10.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  10.00-11.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  11.00-12.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  12.00-13.00  sec  2.39 MBytes  20.0 Mbits/sec  1728
[  5]  13.00-14.00  sec  2.38 MBytes  20.0 Mbits/sec  1725
[  5]  14.00-15.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  15.00-16.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  16.00-17.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  17.00-18.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
[  5]  18.00-19.00  sec  2.38 MBytes  20.0 Mbits/sec  1727
[  5]  19.00-20.00  sec  2.38 MBytes  20.0 Mbits/sec  1726
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-20.00  sec  47.7 MBytes  20.0 Mbits/sec  0.000 ms  0/34529 (0%)  sender
[  5]   0.00-21.28  sec  24.5 MBytes  9.66 Mbits/sec  1.827 ms  16773/34527 (49%)  receiver

iperf Done.
```

Figura 8 - Output do comando iperf mostrando o tráfego UDP entre os dois hosts PC1->PC3 e PC1->PC4

a) Preencha uma pequena tabela com os valores obtidos (saltos, RTT médio, perdas %, duplicados %, débito (Mbps)).

Teste	Saltos	RTT médio (ms)	Perdas (%)	Duplicados (%)	Débito (Mbps)
Ping PC1->PC2	**3**	**303,632**	**10**	**30**	**-**
Ping PC1->PC3	**4**	**404,040**	**16,67**	**0**	**-**
Ping PC1->PC4	**3**	**1,096**	**0**	**0**	**-**
iperf TCP (PC1->PC3)	**-**	**-**	**-**	**-**	**0,0726 (recetor)**
iperf TCP (PC2->PC1)	**-**	**-**	**-**	**-**	**0,107 (recetor)**
iperf TCP (PC1->PC4)	**-**	**-**	**-**	**-**	**9,52 (recetor)**
iperf UDP (PC1->PC3)	**-**	**-**	**95,3**	**-**	**0,729 (recetor)**
iperf UDP (PC1->PC4)	**-**	**-**	**49**	**-**	**9,66 (recetor)**
iperf UDP (PC2->PC1)	**-**	**-**	**95**	**-**	**0,897 (recetor)**

Tabela 1 - Tabela com os valores obtidos nas figuras anteriores

1.3 Testes TCP e UDP em condições adversas

a) Compare os resultados obtidos (débito, perdas vistas pela aplicação, estabilidade).

No teste TCP, o débito medido no recetor foi de 107 Kbit/s, com 22 retransmissões e vários segundos a 0 bit/s. No teste UDP, o débito no recetor foi de 897 Kbit/s, estável em cerca de 973 Kbit/s por segundo. O UDP obteve assim um débito muito superior e mais estável.

Quanto às perdas vistas pela aplicação, o TCP não as expõe, porque as recupera por retransmissão. No UDP, o servidor registou 95% de datagramas perdidos (32760 em 34517), enquanto o cliente reportou 0% de perdas, por não receber confirmações.

b) Como reagem o UDP e o TCP às perdas e ao excesso de tráfego? Discuta as vantagens e desvantagens de cada protocolo, dando exemplos de aplicações que beneficiem mais de um ou do outro.

O TCP retransmite os segmentos perdidos e reduz a janela de tráfego, diminuindo o ritmo de envio. Isto garante a entrega, mas com um débito muito baixo numa ligação com perdas e RTT elevado. O UDP não reage, manteve o envio a 20 Mbit/s numa ligação de 1 Mbps, e o excesso foi descartado na rede.

O TCP tem como vantagens a entrega fiável e ordenada e o controlo de trafego, e como desvantagens o menor débito com perdas e o maior overhead. O UDP tem como vantagens o baixo overhead e a baixa latência, e como desvantagem a ausência de garantias de entrega, ficando a fiabilidade a cargo da aplicação.

Beneficiam mais do TCP aplicações como a web (HTTP), a transferência de ficheiros (FTP) e o acesso remoto (SSH). Beneficiam mais do UDP aplicações como VoIP, videoconferência e jogos online.

Parte II: Transferência de ficheiros

2.1 Transfira os dois ficheiros file1 e file2 a partir do PC3 (descarga do PC1 para o PC3) via TFTP e FTP.

```
root@PC1:/tmp/pycore.40591/PC1.conf# mkdir -p /srv/ftp
root@PC1:/tmp/pycore.40591/PC1.conf# usermod -d /srv/ftp ftp
usermod: no changes
<onf# sh -c "yes 'A' | head -c 10K > /srv/ftp/file1"
<dd if=/dev/urandom of=/srv/ftp/file2 bs=1K count=10
10+0 records in
10+0 records out
10240 bytes (10 kB, 10 KiB) copied, 0.00282947 s, 3.6 MB/s
root@PC1:/tmp/pycore.40591/PC1.conf# ls -l /srv/ftp
total 24
-rw-rw-rw- 1 root root 10240 Oct  4 11:35 file1
-rw-rw-rw- 1 root root 10240 Oct  4 11:35 file2
root@PC1:/tmp/pycore.40591/PC1.conf# chmod a-w /srv/ftp
<-osecure_chroot_dir=/srv/ftp -oanonymous_enable=YES
```

```
vcmd
root@PC3:/tmp/pycore.40591/PC3.conf# ftp 10.0.4.10
Connected to 10.0.4.10.
220 (vsFTPd 3.0.3)
Name (10.0.4.10:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> get file1
local: file1 remote: file1
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for file1 (10240 bytes).
226 Transfer complete.
10240 bytes received in 0.45 secs (22.3539 kB/s)
ftp> get file2
local: file2 remote: file2
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for file2 (10240 bytes).
226 Transfer complete.
10240 bytes received in 0.00 secs (41.2052 MB/s)
ftp> quit
221 Goodbye.
root@PC3:/tmp/pycore.40591/PC3.conf#
```

Figura 9 - Criação dos ficheiros e transferência para PC3 via FTP

```
root@PC3:/tmp/pycore.40591/PC3.conf# rm -f file1 file2
root@PC3:/tmp/pycore.40591/PC3.conf# atftp 10.0.4.10
tftp> timeout 10
tftp> get file1
tftp> get file2
timeout: retrying...
tftp> quit
root@PC3:/tmp/pycore.40591/PC3.conf# ls -l file1 file2
-rw-rw-rw- 1 root root 10240 Oct  4 12:42 file1
-rw-rw-rw- 1 root root 10240 Oct  4 12:43 file2
root@PC3:/tmp/pycore.40591/PC3.conf#
```

Figura 10 - Transferência dos ficheiros para PC3 via TFTP

A transferência dos ficheiros decorreu com sucesso em ambos os protocolos, assegurando o tamanho integral de 10240 bytes em file1 e file2.

No FTP (sobre TCP), o processo foi imediato e transparente, acusando conclusão em 0.45 s para o primeiro ficheiro e reportando débitos de 22.35 KB/s e 41.20 MB/s.

No TFTP (sobre UDP), a ausência de fiabilidade no transporte obrigou a configurar timeout 10 na aplicação para contornar as perdas da rota, sendo visível a recuperação por retentativa (timeout: retrying...) antes de terminar a receção.

a) Identifique qual a rota que foi usada na transferência dos dados

```
root@PC3:/tmp/pycore.40591/PC3.conf# traceroute -n 10.0.4.10
traceroute to 10.0.4.10 (10.0.4.10), 30 hops max, 60 byte packets
 1  10.0.7.1  0.346 ms  0.348 ms  0.296 ms
 2  10.0.1.1  101.566 ms *  102.580 ms
 3  * 10.0.0.1  403.977 ms  404.356 ms
 4  * * *
 5  * 10.0.4.10  405.833 ms  406.406 ms
```

Figura 11 - Comprovativo da rota usada

A rota utilizada na transferência de ficheiros entre o PC3 e o PC1 atravessa o caminho PC3 → n3 (10.0.7.1) → n2 (10.0.1.1) → n1 (10.0.0.1) → PC1 (10.0.4.10). O percurso totaliza 4 saltos na rede com um RTT final em torno de 405 ms. A presença de perdas intermédias visíveis nos tempos de resposta assinalados com asterisco reflete a acumulação das taxas de perda e atraso configuradas nos enlaces n2–n3 e n1–n2.

b) Apresente um diagrama temporal para a transferência do file1 por FTP. Uma vez que o FTP criar mais que uma conexão de transporte em simultâneo, normalmente uma para comandos e outra para os dados, foque-se apenas na transferência de dados [ftp-data] e não na conexão de controlo. Identifique no diagrama as fases de início de conexão, transferência de dados e fim de conexão, caso existam.

tcp.stream eq 1

No.	Time	Source	Destination	Protocol	Length	Info
57	36.961187076	10.0.4.10	10.0.7.10	TCP	74	20 → 34309 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=…
58	36.961222034	10.0.7.10	10.0.4.10	TCP	74	34309 → 20 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PE…
59	37.964212559	10.0.4.10	10.0.7.10	TCP	74	[TCP Retransmission] 20 → 34309 [SYN] Seq=0 Win=64240 Len=0 MSS=14…
60	37.964251105	10.0.7.10	10.0.4.10	TCP	74	[TCP Retransmission] 34309 → 20 [SYN, ACK] Seq=0 Ack=1 Win=65160 L…
62	38.367311077	10.0.4.10	10.0.7.10	TCP	66	20 → 34309 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1419463647 TSec…
65	38.387375598	10.0.4.10	10.0.7.10	FTP-DA…	1514	FTP Data: 1448 bytes (PORT) (PORT 10,0,7,10,134,5)
66	38.387428095	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [ACK] Seq=1 Ack=1449 Win=63744 Len=0 TSval=2199622061 T…
67	38.399642582	10.0.4.10	10.0.7.10	FTP-DA…	1514	FTP Data: 1448 bytes (PORT) (PORT 10,0,7,10,134,5)
68	38.399672799	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [ACK] Seq=1 Ack=2897 Win=62720 Len=0 TSval=2199622073 T…
69	38.412766013	10.0.4.10	10.0.7.10	FTP-DA…	1514	FTP Data: 1448 bytes (PORT) (PORT 10,0,7,10,134,5)
70	38.412812832	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [ACK] Seq=1 Ack=4345 Win=62720 Len=0 TSval=2199622086 T…
71	38.423800057	10.0.4.10	10.0.7.10	FTP-DA…	1514	FTP Data: 1448 bytes (PORT) (PORT 10,0,7,10,134,5)
72	38.423838138	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [ACK] Seq=1 Ack=5793 Win=62720 Len=0 TSval=2199622097 T…
73	38.435761053	10.0.4.10	10.0.7.10	FTP-DA…	1514	FTP Data: 1448 bytes (PORT) (PORT 10,0,7,10,134,5)
74	38.435795237	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [ACK] Seq=1 Ack=7241 Win=62720 Len=0 TSval=2199622109 T…
75	38.447937513	10.0.4.10	10.0.7.10	FTP-DA…	1514	FTP Data: 1448 bytes (PORT) (PORT 10,0,7,10,134,5)
76	38.447971076	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [ACK] Seq=1 Ack=8689 Win=62720 Len=0 TSval=2199622121 T…
77	38.460228665	10.0.4.10	10.0.7.10	FTP-DA…	1514	FTP Data: 1448 bytes (PORT) (PORT 10,0,7,10,134,5)
78	38.460273865	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [ACK] Seq=1 Ack=10137 Win=62720 Len=0 TSval=2199622134 …
79	38.460493497	10.0.4.10	10.0.7.10	FTP-DA…	170	FTP Data: 104 bytes (PORT) (PORT 10,0,7,10,134,5)
80	38.460515191	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [ACK] Seq=1 Ack=10241 Win=64128 Len=0 TSval=2199622134 …
83	40.875118421	10.0.4.10	10.0.7.10	TCP	66	20 → 34309 [FIN, ACK] Seq=10241 Ack=1 Win=64256 Len=0 TSval=141946…
84	40.875483637	10.0.7.10	10.0.4.10	TCP	66	34309 → 20 [FIN, ACK] Seq=1 Ack=10242 Win=64128 Len=0 TSval=219962…
89	41.889557099	10.0.7.10	10.0.4.10	TCP	66	[TCP Retransmission] 34309 → 20 [FIN, ACK] Seq=1 Ack=10242 Win=641…
91	42.292700278	10.0.4.10	10.0.7.10	TCP	66	20 → 34309 [ACK] Seq=10242 Ack=2 Win=64256 Len=0 TSval=1419467573 …

Figura 12 - Captura com o filtro pedido no enunciado

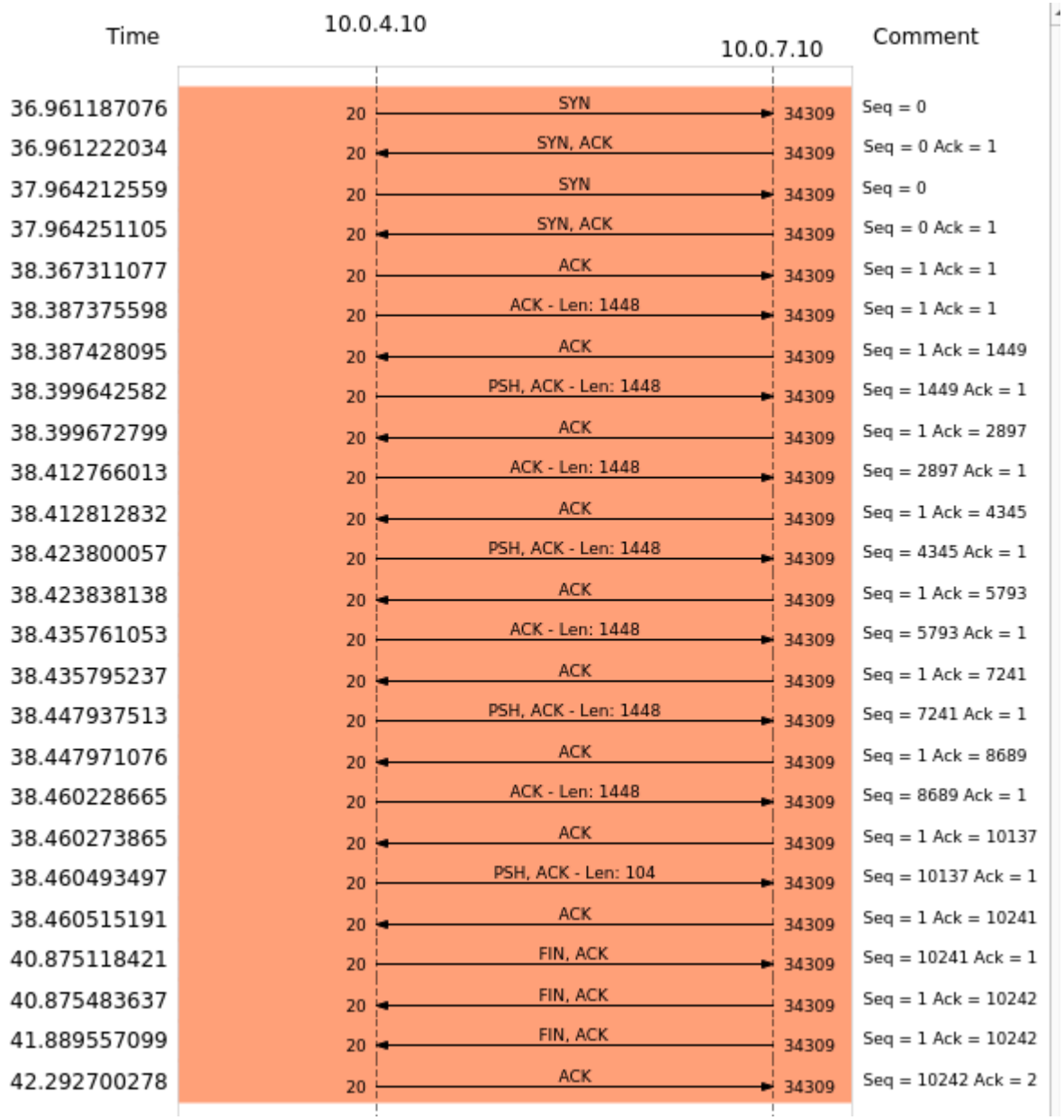


Figura 13 - Diagrama temporal para a transferência do file1 por FTP

O diagrama temporal da transferência do ficheiro file1 via FTP delimita com precisão as três fases da ligação TCP de dados estabelecida entre o servidor (porto 20) e o cliente (porto 34309).

A fase de início de conexão decorre entre os instantes 36.961 s e 38.367 s, onde é executado o three-way handshake com a troca de segmentos SYN, SYN-ACK e ACK, notando-se uma retransmissão do SYN provocada pelas perdas da ligação.

A fase de transferência de dados desenvolve-se entre 38.387 s e 38.460 s, caracterizada pelo fluxo contínuo de segmentos de 1448 bytes e um segmento final com a flag PSH de 104 bytes, somando os 10240 bytes integrais do ficheiro, com confirmações cumulativas progressivas do cliente até Ack=10241.

Por fim, a fase de fim de conexão compreende o intervalo de 40.875 s a 42.292 s, onde se processa a libertação ordenada dos recursos da camada de transporte através do intercâmbio de mensagens FIN, ACK e do ACK final em ambos os sentidos.

c) Apresente um diagrama temporal para a transferência do file1 por TFTP. Identifique, se aplicável, as fases de início de conexão, transferência de dados e fim de conexão. Identifique também os tipos de segmentos trocados e os números de sequência usados quer nos dados como nas confirmações.



Figura 14 - Diagrama temporal para a transferência do file1 por TFTP

Por assentar em UDP, a transferência por TFTP não possui fases de início nem de fim de conexão na camada de transporte, limitando-se ao nível aplicacional a um pedido de leitura inicial (Read Request) do cliente ao porto 69 do servidor e ao término implícito com o último bloco.

A fase de transferência de dados decorre em regime de paragem e espera (stop-and-wait), onde são trocados datagramas do tipo DATA (enviados pelo servidor através do porto 45553) e do tipo Acknowledgement/ACK (enviados pelo cliente através do porto 47894).

O controlo sequencial é feito por números de bloco inteiros em vez de contagem de bytes: o servidor transmite Data Packet, Block: X (com blocos de 512 bytes) e o cliente confirma individualmente cada receção com Acknowledgement, Block: X, progredindo ordenadamente do bloco 1 em diante.

d) De que forma as perdas de pacotes afetaram o desempenho das aplicações nos dois casos anteriores? Qual a camada que lidou com as perdas: transporte ou aplicação? Responda com base nas experiências feitas e nos resultados observados.

No caso do FTP, as perdas afetaram o desempenho de forma quase impercetível para o utilizador, mantendo uma transferência rápida (concluída em 0.45 s). Quem lidou com as perdas foi a camada de transporte (TCP) de forma totalmente transparente à aplicação, através do seu controlo de fluxo e retransmissões automáticas baseadas em ACKs e temporizadores adaptativos.

No caso do TFTP, as perdas degradaram severamente o desempenho, provocando sucessivos bloqueios e fazendo com que a aplicação abortasse de imediato com a configuração padrão devido ao esgotamento do tempo de espera. Quem lidou com as perdas foi exclusivamente a camada de aplicação, dado que o UDP não oferece qualquer mecanismo de fiabilidade. A recuperação de erros dependeu inteiramente do cliente TFTP (atftp), que obrigou a aumentar o temporizador aplicacional (timeout 10) para reenviar os pedidos e pacotes em modo stop-and-wait após cada pacote perdido.

2.2 Transfira os dois ficheiros file1 e file2 a partir do PC4 (descarga do PC1 para o PC4) via TFTP, FTP e HTTP.

a) Relativamente a esta transferência HTTP, identifique:
• O início e o fim da sessão TCP e os números de sequência e de confirmação (ACK) usados na conexão;
• O número de sequência inicial e a forma como é incrementado com cada pacote tanto pelo cliente quanto pelo servidor.

O início da sessão TCP realiza-se nos pacotes 19 a 21 através da troca de segmentos SYN (Seq = 0), SYN-ACK (Seq = 0, Ack = 1) e ACK (Seq = 1, Ack = 1), enquanto o fim da sessão ocorre nos pacotes 41 e 42 com o envio de FIN, ACK pelo cliente (Seq = 79, Ack = 10445) e a confirmação final por ACK do servidor (Seq = 10445, Ack = 80). O número de sequência inicial relativo adotado por ambos os intervenientes é 0.

O incremento do número de sequência pelo cliente dá-se no envio do pedido HTTP de 78 bytes no pacote 22, transitando de 1 para Seq = 79 e mantendo-se inalterado nos pacotes subsequentes por conterem apenas confirmações sem carga útil (Len = 0).

Pelo servidor, o valor progride de 1 para 204 após os primeiros 203 bytes transmitidos no pacote 24 e, subsequentemente, incrementa 1448 unidades a cada segmento de dados enviado (Len = 1448), refletindo de forma estrita o tamanho da carga útil de cada pacote de dados.

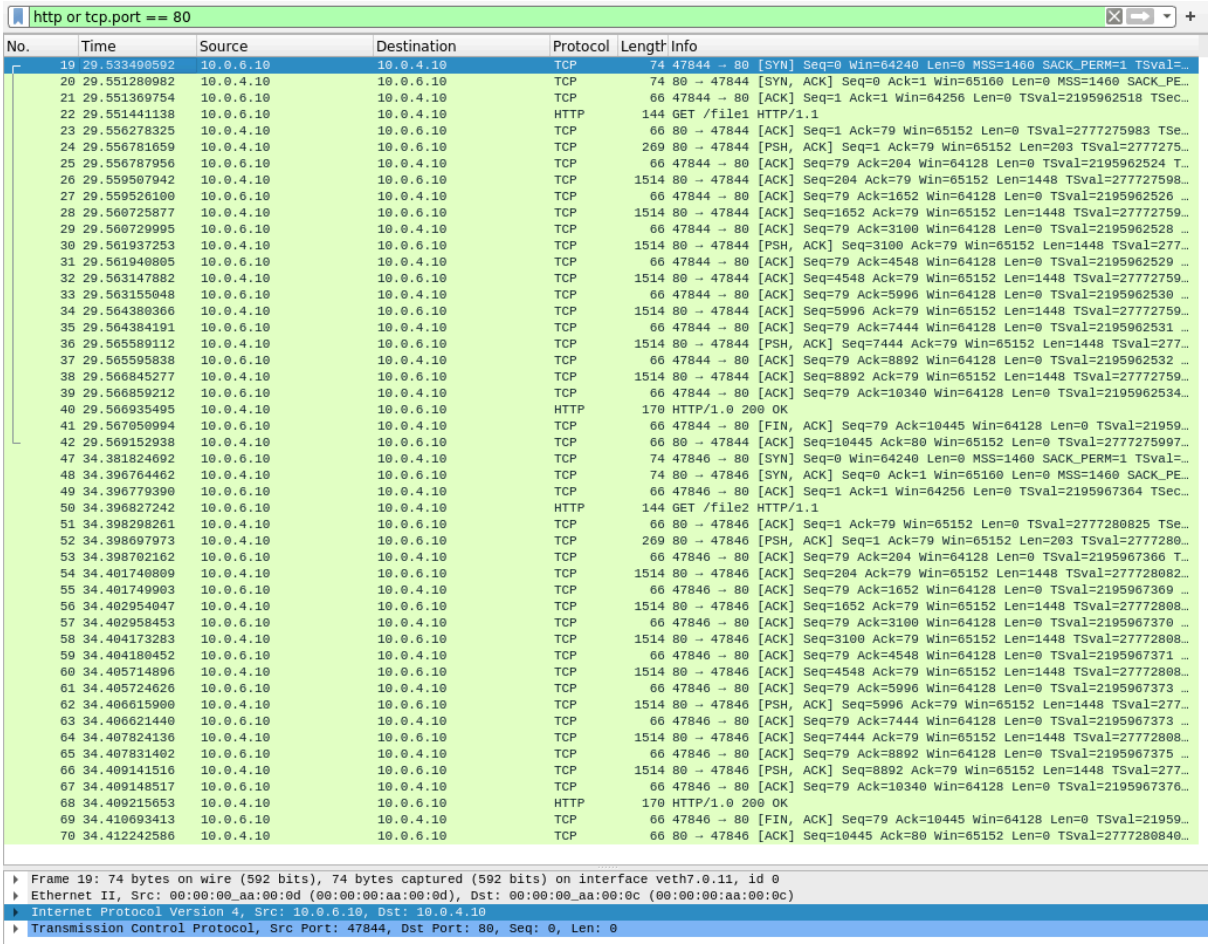


Figura 15 - Captura da transferência http

b) Com base nas transferências efetuadas, qual dos protocolos aplicacionais experimentados (TFTP, FTP e HTTP) seria o mais adequado para a obtenção dos ficheiros pelo PC4 (descarga do PC1 para o PC4)? Justifique.

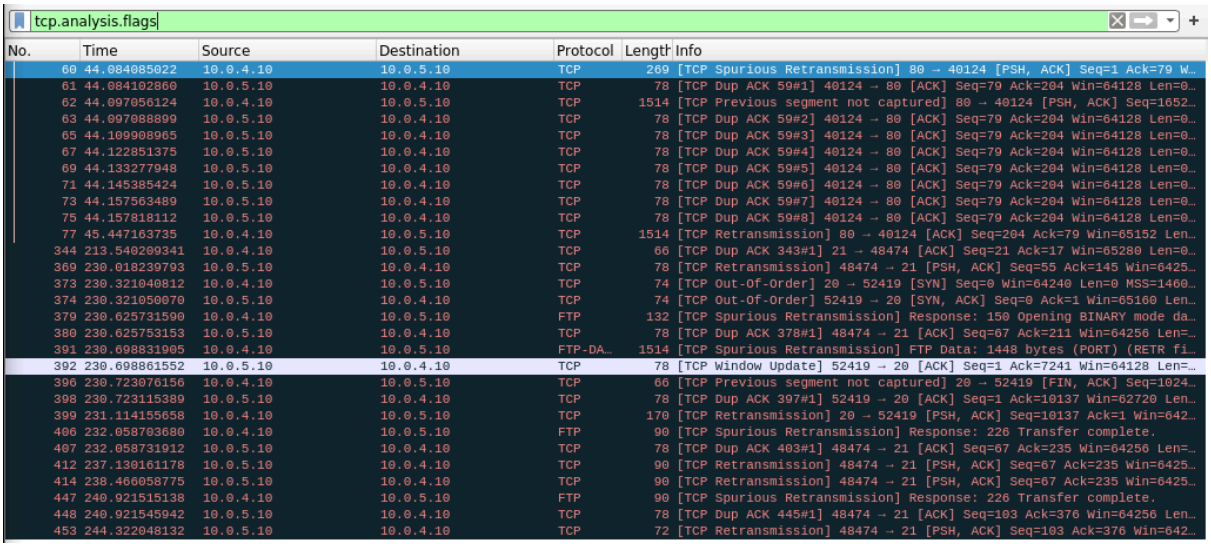
O protocolo mais adequado para a obtenção dos ficheiros pelo PC4 é o HTTP. Face ao TFTP, o HTTP opera sobre TCP, beneficiando de controlo de fluxo e de retransmissões automáticas na camada de transporte, ao passo que o TFTP recorre a UDP em regime estrito de stop-and-wait (blocos de 512 bytes), tornando-se lento e altamente vulnerável a perdas de pacotes e à latência do canal.

Em comparação com o FTP, embora ambos tirem partido da fiabilidade do TCP, o HTTP estabelece uma única ligação TCP sobre o porto 80 para transferir os recursos com pedidos diretos GET, evitando o overhead, a sobrecarga de sinalização e a complexidade de gerir dois canais separados (canal de controlo no porto 21 e canal de dados no porto 20).

2.3 Transfira os dois ficheiros file1 e file2 a partir do PC2 (descarga do PC1 para o PC2) via TFTP, FTP e HTTP.

```
root@PC2:/tmp/pycore.46881/PC2.conf# curl -O http://10.0.4.10/file1
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 10240  100 10240    0     0  14628      0 --:--:-- --:--:-- --:--:-- 14607
root@PC2:/tmp/pycore.46881/PC2.conf# curl -O http://10.0.4.10/file2
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 10240  100 10240    0     0   5179      0  0:00:01  0:00:01 --:--:--  5176
root@PC2:/tmp/pycore.46881/PC2.conf# atftp --option "timeout 10" -g -r file1 10.0.4.10
Option timeout = 10
timeout: retrying...
timeout: retrying...
timeout: retrying...
root@PC2:/tmp/pycore.46881/PC2.conf# atftp --option "timeout 10" -g -r file2 10.0.4.10
Option timeout = 10
timeout: retrying...
timeout: retrying...
root@PC2:/tmp/pycore.46881/PC2.conf# ftp 10.0.4.10
Connected to 10.0.4.10.
220 (vsFTPd 3.0.3)
Name (10.0.4.10:root): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> bin
200 Switching to Binary mode.
ftp> get file1
local: file1 remote: file1
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for file1 (10240 bytes).
226 Transfer complete.
10240 bytes received in 0.49 secs (20.3886 kB/s)
ftp> get file2
local: file2 remote: file2
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for file2 (10240 bytes).
226 Transfer complete.
10240 bytes received in 0.09 secs (115.4295 kB/s)
ftp> quit
221 Goodbye.
root@PC2:/tmp/pycore.46881/PC2.conf# ls -l file1 file2
-rw-rw-rw- 1 root root 10240 Oct  4 15:18 file1
-rw-rw-rw- 1 root root 10240 Oct  4 15:18 file2
```

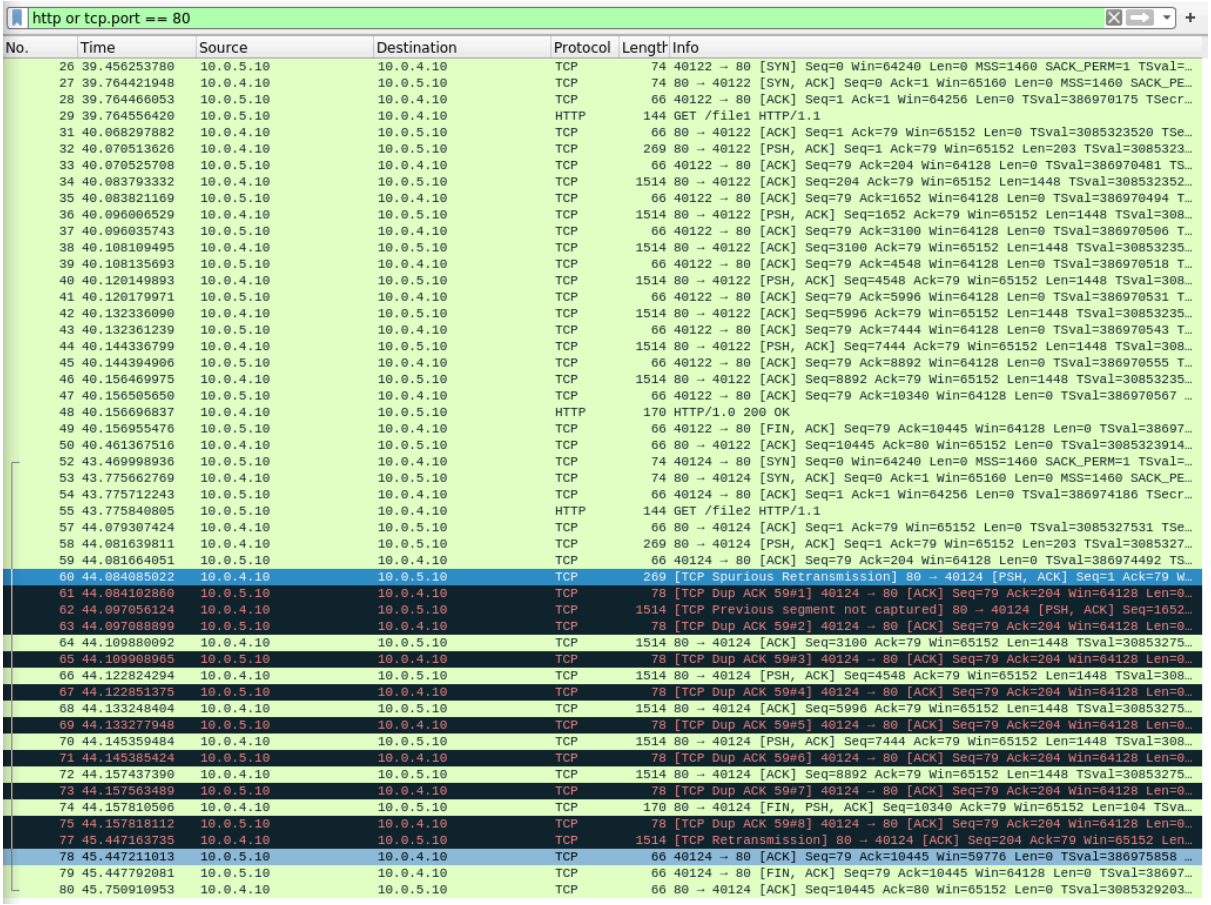
Figura 16 - Comandos no terminal para a captura dos pacotes pedidos no enunciado



tcp.analysis.flags

No.	Time	Source	Destination	Protocol	Length	Info
60	44.084085022	10.0.4.10	10.0.5.10	TCP	269	[TCP Spurious Retransmission] 80 → 40124 [PSH, ACK] Seq=1 Ack=79 W…
61	44.084102860	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#1] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
62	44.097056124	10.0.4.10	10.0.5.10	TCP	1514	[TCP Previous segment not captured] 80 → 40124 [PSH, ACK] Seq=1652…
63	44.097088899	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#2] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
65	44.109908965	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#3] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
67	44.122851375	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#4] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
69	44.133277948	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#5] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
71	44.145385424	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#6] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
73	44.157563489	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#7] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
75	44.157818112	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#8] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
77	45.447163735	10.0.4.10	10.0.5.10	TCP	1514	[TCP Retransmission] 80 → 40124 [ACK] Seq=204 Ack=79 Win=65152 Len…
344	213.540209341	10.0.4.10	10.0.5.10	TCP	66	[TCP Dup ACK 343#1] 21 → 48474 [ACK] Seq=21 Ack=17 Win=65280 Len=0…
369	230.018239793	10.0.5.10	10.0.4.10	TCP	78	[TCP Retransmission] 48474 → 21 [PSH, ACK] Seq=55 Ack=145 Win=6425…
373	230.321040812	10.0.4.10	10.0.5.10	TCP	74	[TCP Out-Of-Order] 20 → 52419 [SYN] Seq=0 Win=64240 Len=0 MSS=1460…
374	230.321050070	10.0.5.10	10.0.4.10	TCP	74	[TCP Out-Of-Order] 52419 → 20 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len…
379	230.625731590	10.0.4.10	10.0.5.10	FTP	132	[TCP Spurious Retransmission] Response: 150 Opening BINARY mode da…
380	230.625753153	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 378#1] 48474 → 21 [ACK] Seq=67 Ack=211 Win=64256 Len=…
391	230.698831905	10.0.4.10	10.0.5.10	FTP-DA…	1514	[TCP Spurious Retransmission] FTP Data: 1448 bytes (PORT) (RETR fi…
392	230.698861552	10.0.5.10	10.0.4.10	TCP	78	[TCP Window Update] 52419 → 20 [ACK] Seq=1 Ack=7241 Win=64128 Len=…
396	230.723076156	10.0.4.10	10.0.5.10	TCP	66	[TCP Previous segment not captured] 20 → 52419 [FIN, ACK] Seq=1024…
398	230.723115389	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 397#1] 52419 → 20 [ACK] Seq=1 Ack=10137 Win=62720 Len…
399	231.114155658	10.0.4.10	10.0.5.10	TCP	170	[TCP Retransmission] 20 → 52419 [PSH, ACK] Seq=10137 Ack=1 Win=642…
406	232.058703680	10.0.4.10	10.0.5.10	FTP	90	[TCP Spurious Retransmission] Response: 226 Transfer complete.
407	232.058731912	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 403#1] 48474 → 21 [ACK] Seq=67 Ack=235 Win=64256 Len=…
412	237.130161178	10.0.5.10	10.0.4.10	TCP	90	[TCP Retransmission] 48474 → 21 [PSH, ACK] Seq=67 Ack=235 Win=6425…
414	238.466058775	10.0.5.10	10.0.4.10	TCP	90	[TCP Retransmission] 48474 → 21 [PSH, ACK] Seq=67 Ack=235 Win=6425…
447	240.921515138	10.0.4.10	10.0.5.10	FTP	90	[TCP Spurious Retransmission] Response: 226 Transfer complete.
448	240.921545942	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 445#1] 48474 → 21 [ACK] Seq=103 Ack=376 Win=64256 Len…
453	244.322048132	10.0.5.10	10.0.4.10	TCP	72	[TCP Retransmission] 48474 → 21 [PSH, ACK] Seq=103 Ack=376 Win=642…

Figura 17 - Perda e duplicação no TCP (HTTP/FTP)



http or tcp.port == 80

No.	Time	Source	Destination	Protocol	Length	Info
26	39.456253780	10.0.5.10	10.0.4.10	TCP	74	40122 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=…
27	39.764421948	10.0.4.10	10.0.5.10	TCP	74	80 → 40122 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PE…
28	39.764466053	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=386970175 TSecr…
29	39.764556420	10.0.5.10	10.0.4.10	HTTP	144	GET /file1 HTTP/1.1
31	40.068297882	10.0.4.10	10.0.5.10	TCP	66	80 → 40122 [ACK] Seq=1 Ack=79 Win=65152 Len=0 TSval=3085323520 TSe…
32	40.070513626	10.0.4.10	10.0.5.10	TCP	269	80 → 40122 [PSH, ACK] Seq=1 Ack=79 Win=65152 Len=203 TSval=3085323…
33	40.070525708	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0 TSval=386970481 TS…
34	40.083793332	10.0.4.10	10.0.5.10	TCP	1514	80 → 40122 [ACK] Seq=204 Ack=79 Win=65152 Len=1448 TSval=308532352…
35	40.083821169	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=79 Ack=1652 Win=64128 Len=0 TSval=386970494 T…
36	40.096006529	10.0.4.10	10.0.5.10	TCP	1514	80 → 40122 [PSH, ACK] Seq=1652 Ack=79 Win=65152 Len=1448 TSval=308…
37	40.096035743	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=79 Ack=3100 Win=64128 Len=0 TSval=386970506 T…
38	40.108109495	10.0.4.10	10.0.5.10	TCP	1514	80 → 40122 [ACK] Seq=3100 Ack=79 Win=65152 Len=1448 TSval=30853235…
39	40.108135693	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=79 Ack=4548 Win=64128 Len=0 TSval=386970518 T…
40	40.120149893	10.0.4.10	10.0.5.10	TCP	1514	80 → 40122 [PSH, ACK] Seq=4548 Ack=79 Win=65152 Len=1448 TSval=308…
41	40.120179971	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=79 Ack=5996 Win=64128 Len=0 TSval=386970531 T…
42	40.132336090	10.0.4.10	10.0.5.10	TCP	1514	80 → 40122 [ACK] Seq=5996 Ack=79 Win=65152 Len=1448 TSval=30853235…
43	40.132361239	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=79 Ack=7444 Win=64128 Len=0 TSval=386970543 T…
44	40.144336799	10.0.4.10	10.0.5.10	TCP	1514	80 → 40122 [PSH, ACK] Seq=7444 Ack=79 Win=65152 Len=1448 TSval=308…
45	40.144394906	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=79 Ack=8892 Win=64128 Len=0 TSval=386970555 T…
46	40.156469975	10.0.4.10	10.0.5.10	TCP	1514	80 → 40122 [ACK] Seq=8892 Ack=79 Win=65152 Len=1448 TSval=30853235…
47	40.156505650	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [ACK] Seq=79 Ack=10340 Win=64128 Len=0 TSval=386970567 …
48	40.156696837	10.0.4.10	10.0.5.10	HTTP	170	HTTP/1.0 200 OK
49	40.156955476	10.0.5.10	10.0.4.10	TCP	66	40122 → 80 [FIN, ACK] Seq=79 Ack=10445 Win=64128 Len=0 TSval=38697…
50	40.461367516	10.0.4.10	10.0.5.10	TCP	66	80 → 40122 [ACK] Seq=10445 Ack=80 Win=65152 Len=0 TSval=3085323914…
52	43.469998936	10.0.5.10	10.0.4.10	TCP	74	40124 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=…
53	43.775662769	10.0.4.10	10.0.5.10	TCP	74	80 → 40124 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PE…
54	43.775712243	10.0.5.10	10.0.4.10	TCP	66	40124 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=386974186 TSecr…
55	43.775840805	10.0.5.10	10.0.4.10	HTTP	144	GET /file2 HTTP/1.1
57	44.079307424	10.0.4.10	10.0.5.10	TCP	66	80 → 40124 [ACK] Seq=1 Ack=79 Win=65152 Len=0 TSval=3085327531 TSe…
58	44.081639811	10.0.4.10	10.0.5.10	TCP	269	80 → 40124 [PSH, ACK] Seq=1 Ack=79 Win=65152 Len=203 TSval=3085327…
59	44.081664051	10.0.5.10	10.0.4.10	TCP	66	40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0 TSval=386974492 TS…
60	44.084085022	10.0.4.10	10.0.5.10	TCP	269	[TCP Spurious Retransmission] 80 → 40124 [PSH, ACK] Seq=1 Ack=79 W…
61	44.084102860	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#1] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
62	44.097056124	10.0.4.10	10.0.5.10	TCP	1514	[TCP Previous segment not captured] 80 → 40124 [PSH, ACK] Seq=1652…
63	44.097088899	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#2] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
64	44.109880092	10.0.4.10	10.0.5.10	TCP	1514	80 → 40124 [ACK] Seq=3100 Ack=79 Win=65152 Len=1448 TSval=30853275…
65	44.109908965	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#3] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
66	44.122824294	10.0.4.10	10.0.5.10	TCP	1514	80 → 40124 [PSH, ACK] Seq=4548 Ack=79 Win=65152 Len=1448 TSval=308…
67	44.122851375	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#4] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
68	44.133248404	10.0.4.10	10.0.5.10	TCP	1514	80 → 40124 [ACK] Seq=5996 Ack=79 Win=65152 Len=1448 TSval=30853275…
69	44.133277948	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#5] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
70	44.145359484	10.0.4.10	10.0.5.10	TCP	1514	80 → 40124 [PSH, ACK] Seq=7444 Ack=79 Win=65152 Len=1448 TSval=308…
71	44.145385424	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#6] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
72	44.157437390	10.0.4.10	10.0.5.10	TCP	1514	80 → 40124 [ACK] Seq=8892 Ack=79 Win=65152 Len=1448 TSval=30853275…
73	44.157563489	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#7] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
74	44.157810506	10.0.4.10	10.0.5.10	TCP	170	80 → 40124 [FIN, PSH, ACK] Seq=10340 Ack=79 Win=65152 Len=104 TSva…
75	44.157818112	10.0.5.10	10.0.4.10	TCP	78	[TCP Dup ACK 59#8] 40124 → 80 [ACK] Seq=79 Ack=204 Win=64128 Len=0…
77	45.447163735	10.0.4.10	10.0.5.10	TCP	1514	[TCP Retransmission] 80 → 40124 [ACK] Seq=204 Ack=79 Win=65152 Len…
78	45.447211013	10.0.5.10	10.0.4.10	TCP	66	40124 → 80 [ACK] Seq=79 Ack=10445 Win=59776 Len=0 TSval=386975858 …
79	45.447792081	10.0.5.10	10.0.4.10	TCP	66	40124 → 80 [FIN, ACK] Seq=79 Ack=10445 Win=64128 Len=0 TSval=38697…
80	45.750910953	10.0.4.10	10.0.5.10	TCP	66	80 → 40124 [ACK] Seq=10445 Ack=80 Win=65152 Len=0 TSval=3085329203…

Figura 18 - Perda e duplicação no TCP (HTTP)

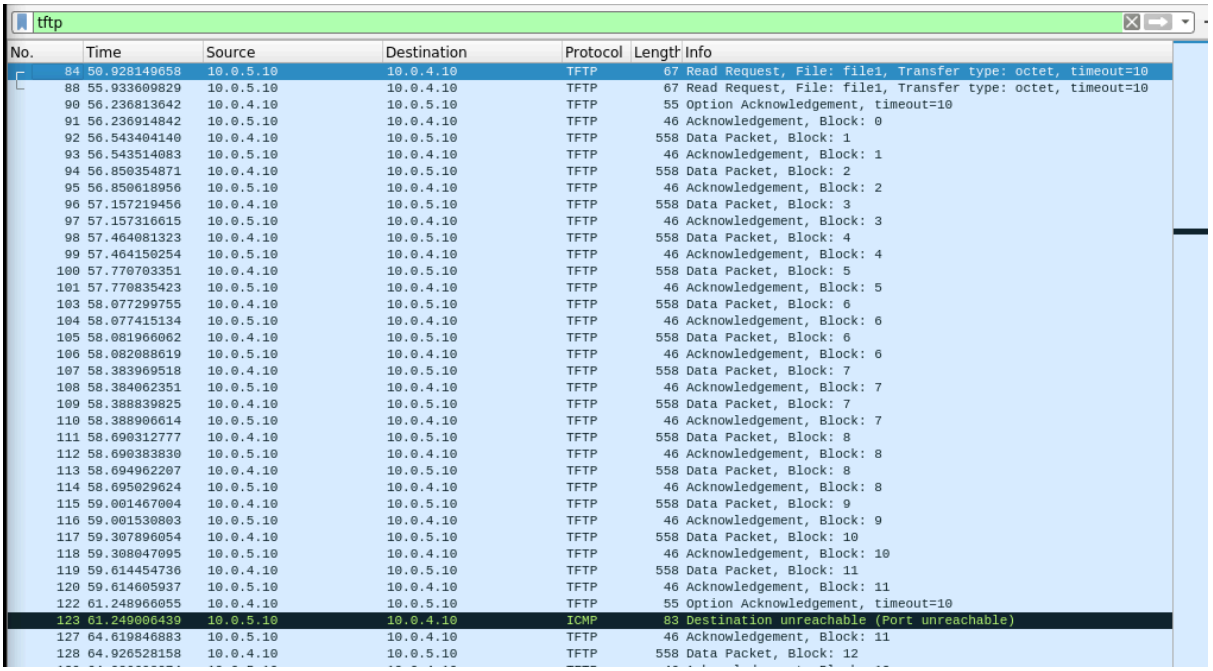


Figura 19 - Perda e duplicação no TCP (TFTP)

a) Relativamente a esta transferência HTTP:
• Identifique a perda e a duplicação de pacotes numa sessão TCP;
• Explique o impacto da perda e duplicação de pacotes numa sessão TCP, bem como os mecanismos usados pelo TCP para lidar com estas situações.

Na transferência HTTP capturada no PC2, a perda de pacotes identifica-se na descarga do segundo ficheiro através do pacote 77 com a marcação [TCP Retransmission] (Seq=204), motivada pelo facto de o cliente não ter recebido esse segmento atempadamente e ter assinalado a receção de pacotes desordenados com [TCP Previous segment not captured] no pacote 62.

Já a duplicação de pacotes evidencia-se pelo segmento redundante do servidor no pacote 60 assinalado como [TCP Spurious Retransmission], bem como pelas múltiplas confirmações repetidas enviadas consecutivamente pelo cliente nos pacotes 61 a 75 marcadas como [TCP Dup ACK] a pedir insistentemente o byte Ack=204 em falta.

O impacto da perda traduz-se no aumento do tempo de transferência e na quebra de débito útil devido à paragem da janela e à redução da taxa de envio, ao passo que a duplicação acarreta desperdício de largura de banda e processamento redundante.

Para lidar com as perdas, o TCP recorre ao temporizador de retransmissão adaptativo (RTO) e à retransmissão rápida (Fast Retransmit) acionada por três ACKs duplicados; para tratar as duplicações, utiliza os números de sequência de cada cabeçalho para detetar e descartar dados repetidos no buffer, reenviando de imediato um ACK atualizado para sincronizar o emissor.

b) Com base nas transferências efetuadas, qual dos protocolos aplicacionais experimentados (TFTP, FTP e HTTP) seria o mais adequado para a obtenção dos ficheiros pelo PC2 (descarga do PC1 para o PC2)? Justifique.

O protocolo mais adequado para a obtenção dos ficheiros pelo PC2 continua a ser o HTTP (seguido pelo FTP).

Face ao TFTP, este último opera sobre UDP num esquema estrito de paragem-e-espera (stop-and-wait bloco a bloco), o que se traduz num desempenho altamente penalizado num enlace instável com perdas e duplicações (como comprovam as retransmissões de blocos e timeouts observados nos pacotes da captura).

Em contrapartida, tanto o HTTP como o FTP assentam sobre TCP, beneficiando de mecanismos robustos de recuperação e controlo na camada de transporte (como retransmissões rápidas e controlo de fluxo/congestão).

Entre o HTTP e o FTP, o HTTP revela-se mais adequado por requerer apenas uma única conexão TCP no porto 80 para descarregar diretamente os recursos via pedidos GET, evitando o overhead, os atrasos e a vulnerabilidade adicional do FTP, que exige gerir e sincronizar dois canais TCP distintos (controlo no porto 21 e dados no porto 20), os quais sofreram múltiplas retransmissões e desfasamentos na presença de anomalias na ligação.

2.4 Prepare e execute uma captura de tráfego de uma transferência TCP realizada, por exemplo, com iperf3 ou curl.
Identifique e explique, com base nos pacotes capturados, dois dos mecanismos internos do TCP:

- **Retransmissão de segmentos;**
- **Controlo de fluxo (janela de receção);**
- **Controlo de congestionamento.**

Mostre evidências na captura (ex.: pacotes repetidos, valores de janela, variação do débito) que suportem a sua explicação.

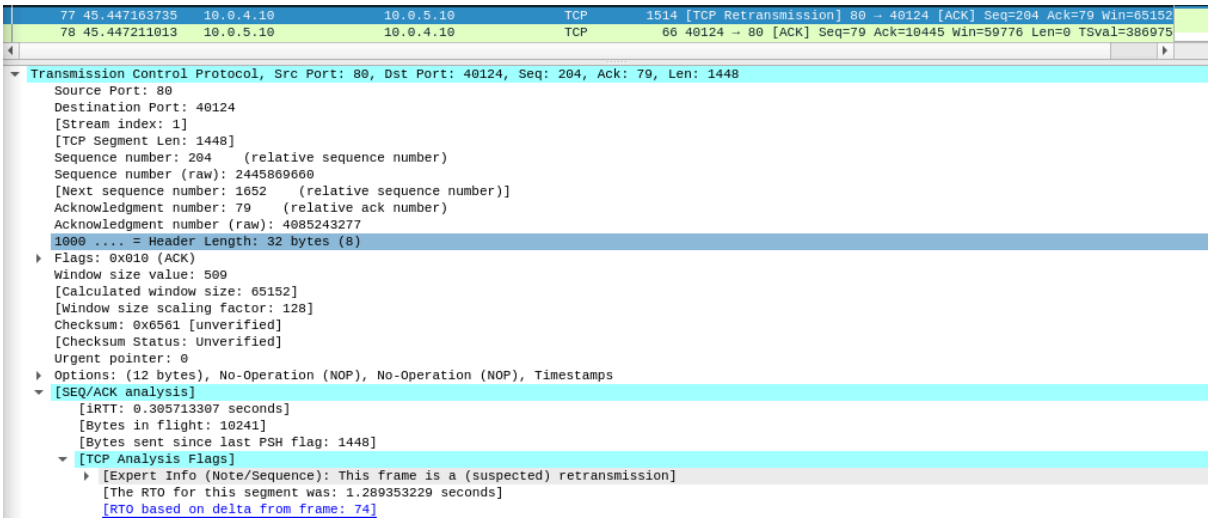


Figura 20 - Evidência para a Retransmissão de Segmentos

A retransmissão de segmentos assegura a fiabilidade na entrega de dados quando ocorrem perdas na rede, sendo gerida por temporizadores adaptativos (RTO) que acionam o reenvio caso a confirmação não chegue atempadamente.

Como evidência na captura, o pacote 77 é identificado pelo Wireshark como [TCP Retransmission] do segmento com Seq=204 e dimensão de 1448 bytes, enviado pelo servidor no porto 80 para o cliente no porto 40124. A inspeção detalhada do TCP no campo [SEQ/ACK analysis] comprova a atuação do mecanismo interno através da anotação [This frame is a (suspected) retransmission] associada ao esgotamento do temporizador de retransmissão registado no campo [The RTO for this segment was: 1.289353229 seconds], o qual disparou o reenvio dos dados após a ausência de confirmação.

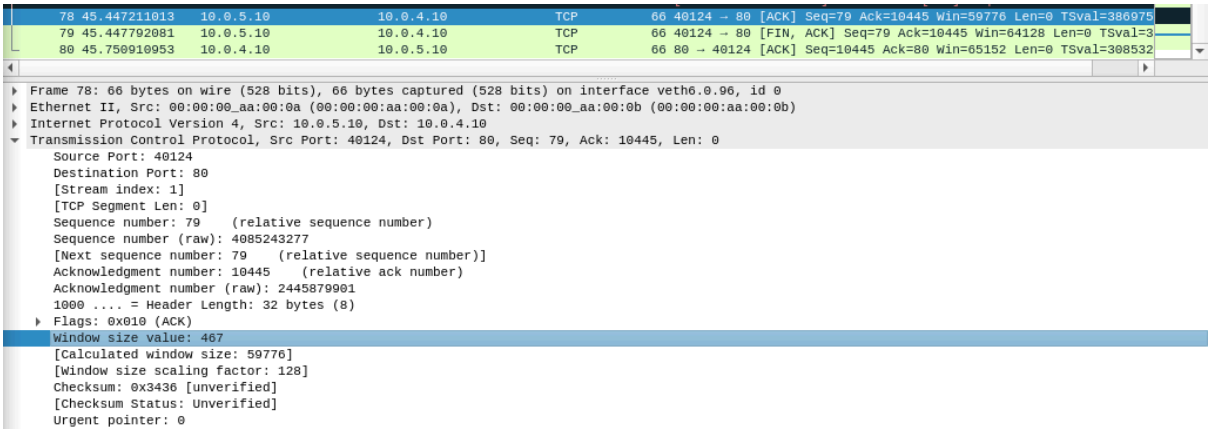


Figura 21 - Evidência para o Controlo de Fluxo

O controlo de fluxo impede a sobrecarga do destinatário, ajustando a taxa de emissão do transmissor ao espaço disponível na memória intermédia de receção através do anúncio dinâmico da janela. Como evidência, a dissecação do pacote 78 (uma confirmação enviada pelo cliente PC2 para o servidor PC1 com Ack=10445) apresenta no cabeçalho TCP o campo Window size value: 467 que, combinado com o fator de escala de janela negociado (Window size scaling factor: 128), resulta no valor efetivo Calculated window size: 59776 bytes. Este valor numérico demonstra o controlo de fluxo ativo na sessão, onde o nó recetor impõe diretamente ao emissor o limite máximo de dados contíguos que este pode colocar na rede sem saturar o seu buffer local.

Parte III: As aplicações e camada de transporte

3.1 – Com base no trabalho realizado, identifique para cada aplicação executada, qual o protocolo de aplicação, o protocolo de transporte, porta de atendimento e overhead de transporte.

Comando usado (aplicação)	Protocolo de Aplicação (se aplicável)	Protocolo de transporte (se aplicável)	Porta de atendimento (se aplicável)	Overhead de Transporte em bytes (se aplicável)
wget, lynx ou via browser	**HTTP**	**TCP**	**80**	**40**
ssh, sftp	**SSH**	**TCP**	**22**	**20**
ftp	**FTP**	**TCP**	**21**	**20**
Tftp	**TFTP**	**UDP**	**69**	**8**
telnet	**Telnet**	**TCP**	**23**	**20**
nslookup ou dig	**DNS**	**UDP**	**53**	**8**
Ping	**n/a**	**n/a**	**n/a**	**n/a**
Traceroute	**n/a**	**UDP**	**33434**	**8**

Tabela 2 - Tabela com os valores obtidos

Os overheads de transporte resultam do cabeçalho do protocolo usado. O TCP tem no mínimo 20 bytes, mas pode ter mais quando são usadas opções, como nos pacotes SYN onde observámos 40 bytes devido a opções como MSS, SACK, Timestamps e Window scale. O UDP tem sempre 8 bytes, que correspondem aos campos de portas de origem e destino, comprimento e checksum. O DNS, o TFTP e as sondas do traceroute usam UDP, enquanto o HTTP, o FTP, o SSH ou SFTP e o Telnet usam TCP. O ping usa o protocolo ICMP, que opera diretamente sobre o IP, por isso não apresenta portas nem protocolo de transporte. Por fim, o FTP utiliza duas ligações TCP, nomeadamente a ligação de controlo na porta 21 e a ligação de dados numa porta dinâmica no modo passivo, onde é o cliente a iniciar a ligação para atravessar a NAT com sucesso.

HTTP: No print que se segue, verifica-se a utilização do protocolo HTTP sobre TCP com destino à porta 80. No pacote 205 (SYN), observa-se o início da ligação TCP a partir da porta 51038, registando um overhead de transporte de 40 bytes devido aos 20 bytes adicionais de opções TCP.

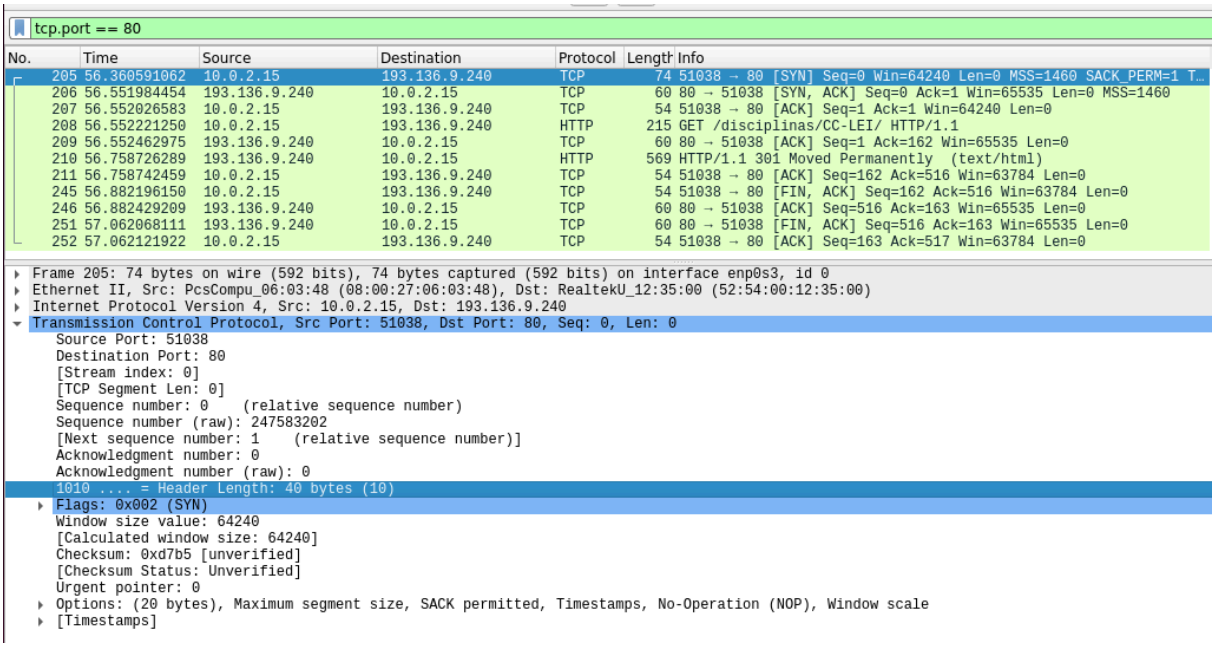


Figura 22 - Captura wget

FTP: No print que se segue, verifica-se a utilização do protocolo FTP sobre TCP com destino à porta de atendimento 21, registando um overhead de transporte de 20 bytes referente ao cabeçalho padrão do protocolo TCP.

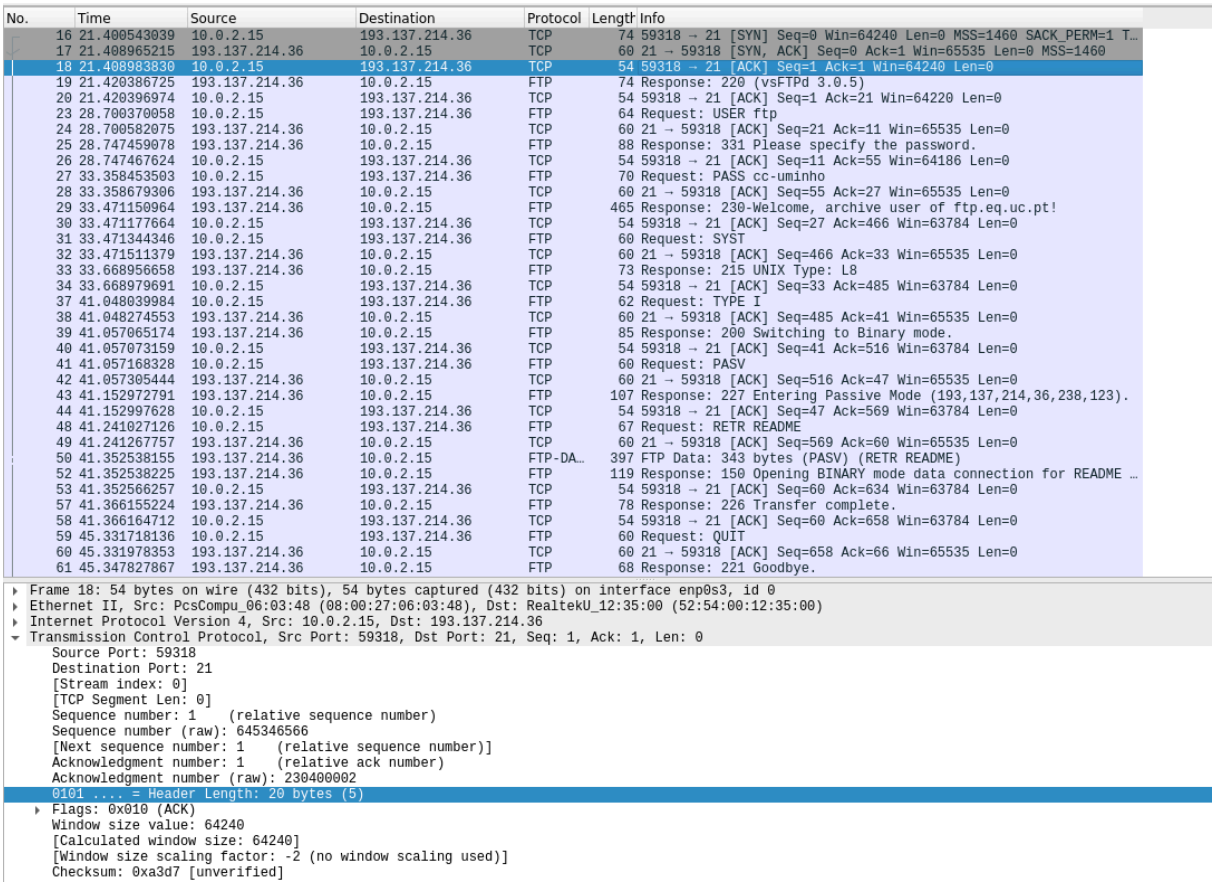


Figura 23 - Captura ftp

TFTP: No print que se segue, verifica-se a utilização do protocolo TFTP sobre UDP com destino à porta de atendimento 69, registando um overhead de transporte de 8 bytes referente ao cabeçalho do protocolo UDP.

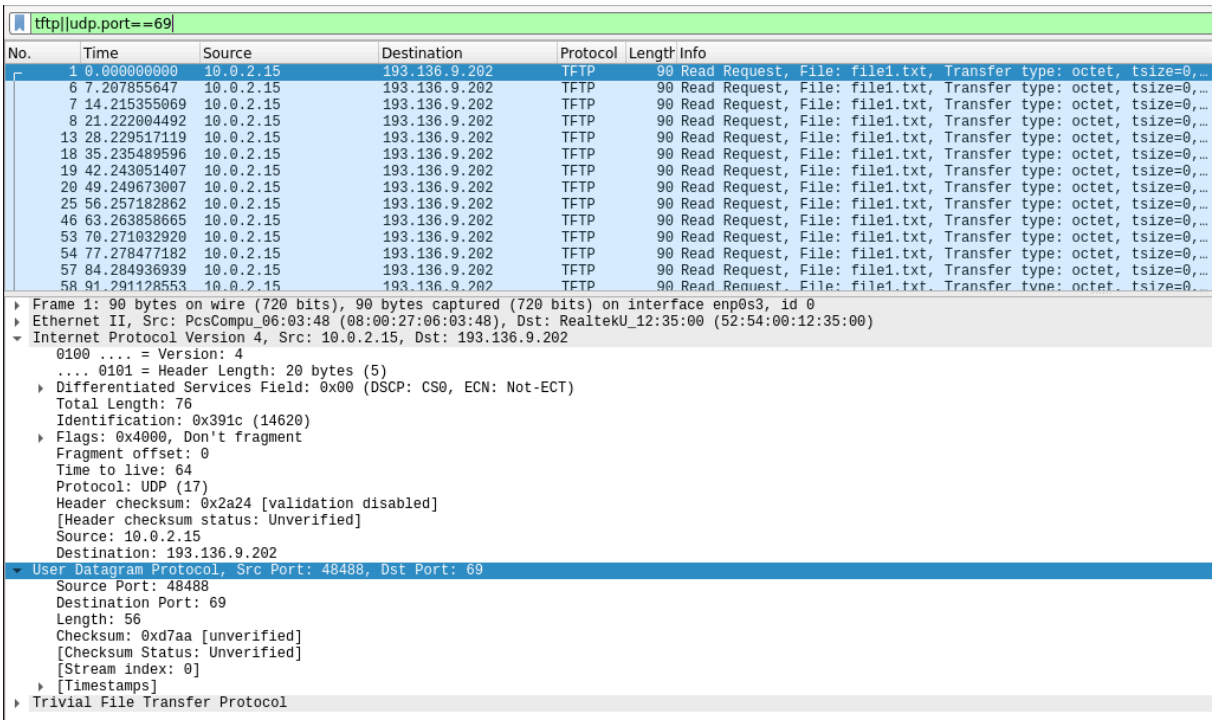


Figura 24 - Captura tftp

SSH: No print que se segue, verifica-se a utilização do protocolo SSH sobre TCP com destino à porta de atendimento 22, registando um overhead de transporte de 20 bytes referente ao cabeçalho padrão do protocolo TCP.

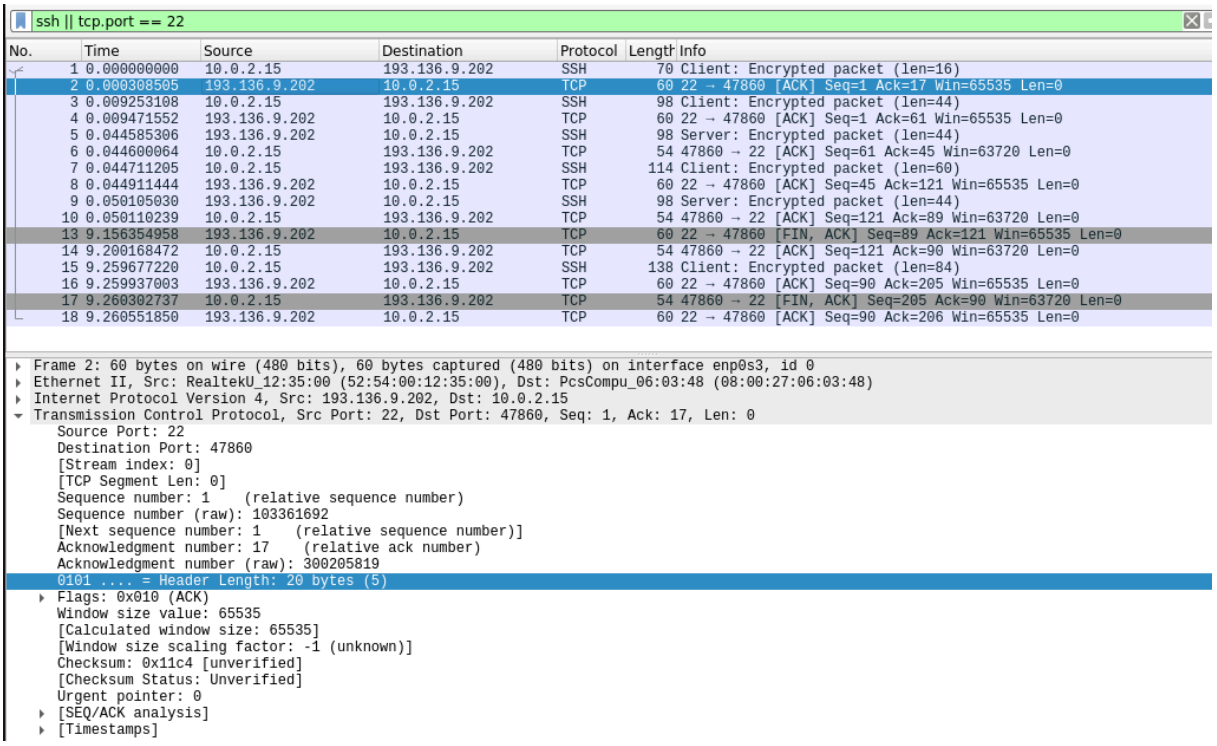


Figura 25 - Captura SSH

Ping: No print que se segue, conseguimos verificar que é utilizado o protocolo ICMP na camada de rede, não existindo porta de acesso nem overhead na camada de transporte, uma vez que o ICMP opera diretamente sobre o protocolo IP com o número de protocolo 1.

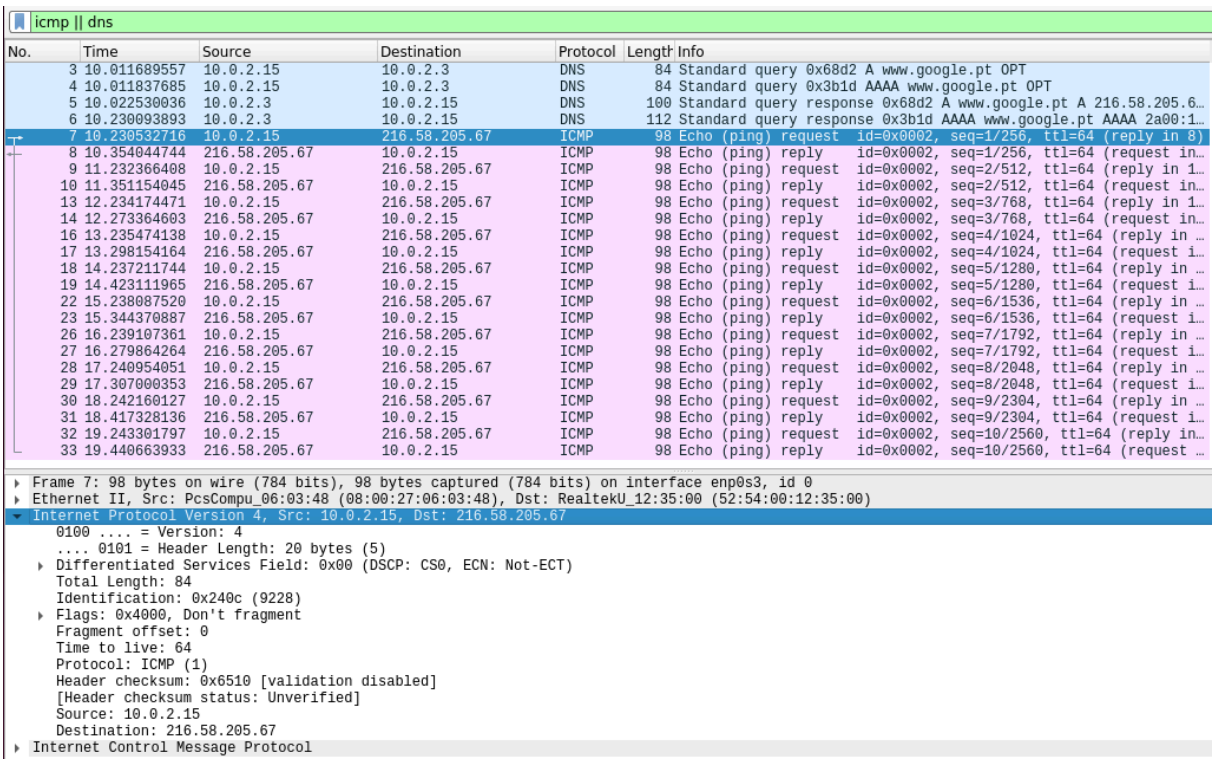


Figura 26 - Captura ping

Traceroute: No print que se segue, conseguimos verificar que é utilizado o protocolo UDP no envio dos pacotes de sonda com destino à porta 33434, apresentando um overhead de transporte de 8 bytes referente ao cabeçalho do protocolo UDP.

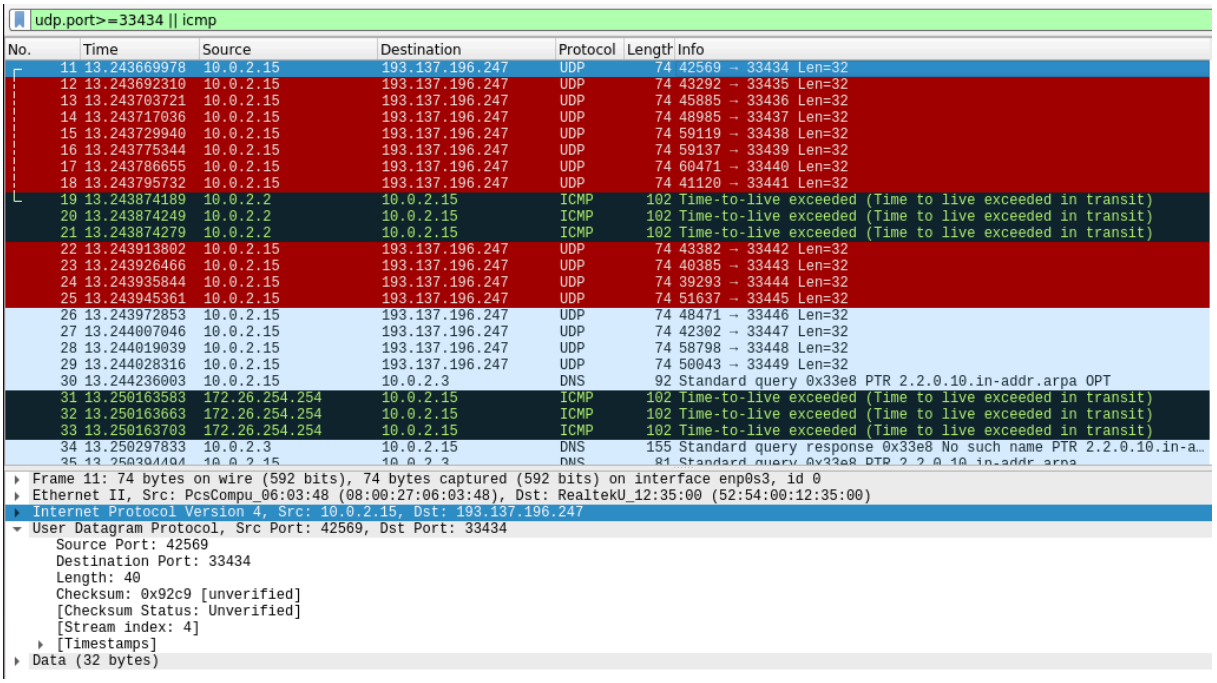


Figura 27 - Captura traceroute

TELNET: No print que se segue, verifica-se a utilização do protocolo Telnet sobre TCP com destino à porta de atendimento 23, registando um overhead de transporte de 20 bytes referente ao cabeçalho padrão do protocolo TCP.

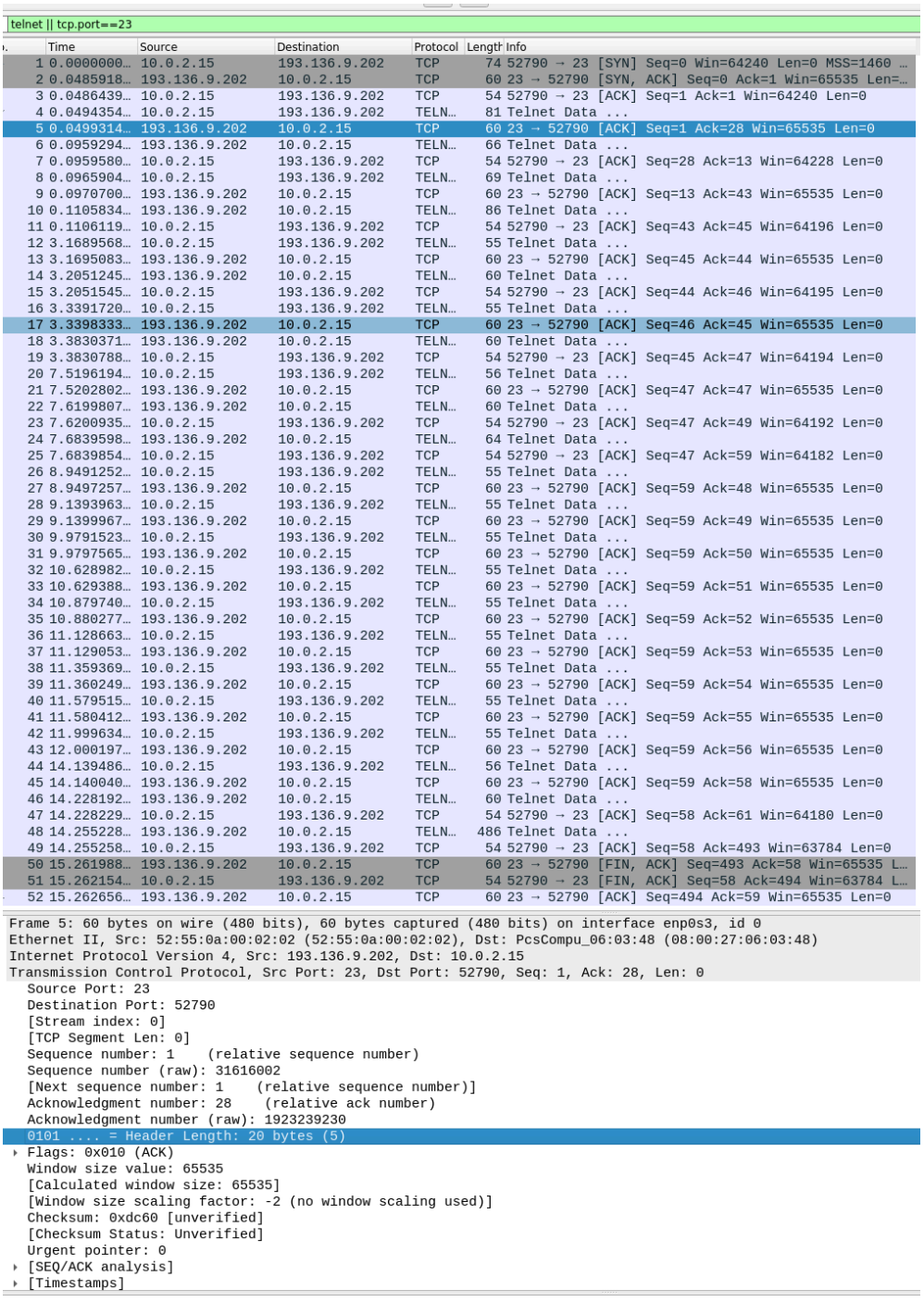


Figura 28 - Captura telnet